Indian Institute of Technology Bombay
Department of Computer Science and Engineering

Benchmarking and Performance Analysis of LLM Inferencing Systems

Systems for LLMs



Saksham Rathi

Under the guidance of

Prof. Mythili Vutukuru

Bachelor's Thesis Project

Academic Year: 2025–26

Contents

List of Tables 3

List of Figures 4

1 Background 6
- 1.1 Stages in the Model Creation Pipeline . . . 7
- 1.2 Transformer Architecture . . . 8
  - 1.2.1 Tokenization . . . 8
  - 1.2.2 Embedding and Attention . . . 8
  - 1.2.3 Multi-Head Masked Attention . . . 8
- 1.3 Transformer Based Large Langauge Models . . . 9
  - 1.3.1 Autoregressive Generation . . . 10
- 1.4 LLM Inferencing . . . 10
  - 1.4.1 Forward Pass . . . 10
  - 1.4.2 Token Probability Distribution . . . 11
  - 1.4.3 Generation Loop . . . 11
  - 1.4.4 Modes of Inference . . . 11
- 1.5 Inference Engines . . . 12
  - 1.5.1 Optimizations in Inference Engines . . . 12
- 1.6 Prefill vs Decode . . . 13
- 1.7 Metrics . . . 13
  - 1.7.1 Throughput . . . 13
  - 1.7.2 Latency . . . 13

2 vLLM 15
- 2.1 Memory Challenges . . . 16
- 2.2 vLLM . . . 16
  - 2.2.1 PagedAttention . . . 16
  - 2.2.2 KV Cache Manager . . . 17
  - 2.2.3 Parallel Sampling . . . 17
  - 2.2.4 Scheduling and Preemption . . . 17
- 2.3 Automatic Prefix Caching in vLLM . . . 18
  - 2.3.1 Components . . . 18
  - 2.3.2 Operations . . . 19

3 SGLang 20
- 3.1 LM Programs . . . 20
- 3.2 Programming Model . . . 20
- 3.3 RadixAttention . . . 21
- 3.4 Compressed Finite State Machine . . . 22

4 FastTree **23**
4.1 Introduction . . . 23
4.2 Tree Structure Adaptive Runtime Optimization . . . 24
4.3 Efficient Tree Structure Attention Kernel Design . . . 26

5 SwiftLLM **27**

6 Matrix Multiplication Analysis **28**

7 Experiments **30**
7.1 System Prompt in Cache Experiment . . . 30
7.1.1 Example 1 . . . 30
7.1.2 Example 2 . . . 31
7.1.3 Observations . . . 32
7.1.4 Analysis over different steps and using different metrics . . . 32
7.1.5 Profiling across pipeline stages . . . 33
7.2 Batch of Requests . . . 33
7.3 Throughput and Latency Measurements across different techniques . . . 33
7.4 LLM Request Timelines . . . 35
7.4.1 Chunked Prefill Enabled . . . 35
7.5 Maximum Throughput with vLLM . . . 40
7.6 Nsight Systems GPU Metrics . . . 40
7.7 Throughput vs Load Comparison For vLLM . . . 42
7.8 Families of Requests . . . 42
7.9 Compute Utilization across various max batch sizes . . . 43
7.10 GPU Memory Breakdown . . . 45
7.11 Merging queries belonging to the same family into a single batch . . . 45
7.12 Comparison of vLLM with SGLANG . . . 54

8 Maths behind LLMs **55**
8.1 Transformer FLOPs . . . 55
8.1.1 FLOPs Calculation . . . 55
8.1.2 Model parameters (TinyLlama-1.1B-Chat) . . . 56
8.1.3 FLOPs Calculation for TinyLlama-1.1B-Chat . . . 56
8.1.4 Efficiency Calculation . . . 56
8.1.5 Efficiency Calculation . . . 56
8.1.6 Summary of the nomenclature used . . . 57

Bibliography **58**

List of Tables

7.1 Profiler Output 32
7.2 Profiling results for various pipeline stages 33
7.3 Measuring the time taken to process a batch of requests using different techniques 33
7.4 Nsight Systems GPU Metrics 41
7.5 GPU Memory Breakdown and KV Cache Usage 45
7.6 Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging 45
7.7 Throughput comparison (in req/s). 54

8.1 Variables used in FLOPs 57

List of Figures

1.1 Adopted from llm . . . 6
1.2 Adopted from USE . . . 7
1.3 Architecture of a Large Language Model (3) . . . 9
1.4 Architecture of an Inference Engine (3) . . . 12
1.5 Prefill vs Decode . . . 13

2.1 Memory layout when serving an LLM with 13B parameters on NVIDIA A100 . . . 15
2.2 vLLM system overview. . . . 16

3.1 SGLANG Frontend Language Model . . . 21
3.2 SGLANG Radix Tree Example . . . 22

4.1 KV cache sharing through Radix Tree . . . 23
4.2 Computation pattern of existing systems . . . 23
4.3 Illustration of two context-queries grouping plans . . . 24
4.4 Greedy Algorithm . . . 25

6.1 SM Active and SM Occupancy across different matrix sizes . . . 29

7.1 Throughput . . . 36
7.2 Average Latency . . . 36
7.3 P95 Latency . . . 36
7.4 20 Requests per second . . . 37
7.5 25 Requests per second . . . 37
7.6 All requests at once . . . 37
7.7 Request Timeline with GPU Metrics . . . 38
7.8 Chunked Prefill Enabled - 100 Requests per second . . . 38
7.9 Chunked Prefill Enabled - All requests at once . . . 39
7.10 Number of batch outputs - chunked prefill enabled . . . 40
7.11 Number of batch outputs - chunked prefill disabled . . . 40
7.12 Throughput vs Load . . . 42
7.13 Throughput vs Load for different models for heavy workloads . . . 42
7.14 SM Active and SM Occupancy across different max batch sizes . . . 43
7.15 Times spent across different batch decompositions . . . 44
7.16 Throughput vs Load without prefix pre-filling . . . 46
7.17 Throughput vs Load with prefix pre-filling . . . 47
7.18 Throughput vs Load with prefix pre-filling with ordered family IDs . . . 48
7.19 Throughput vs Load for 1 family . . . 49
7.20 Throughput vs Load for 5 families . . . 50
7.21 Throughput vs Load for 10 families . . . 51
7.22 Throughput vs Load for 100 families . . . 52

7.23 Throughput vs Max Batch Size for multiple families 53
7.24 SM Activity, SM Occupancy and Tensor Core Utilization for different family sizes at max batch size 500 53
7.25 SM Activity, SM Occupancy and Tensor Core Utilization for different family sizes and max batch sizes 54

8.1 FLOPs Calculation Breakdown (from Casson (2023)) 56

Chapter 1

Background

Language models (LMs) are statistical models that predict the next word in a sequence given the previous words. They basically give us a probability distribution over words based on the context provided by the preceeding words. They enable machines to understand and generate human language. They can be used for a lot of tasks like answering questions, text summarization, translation, code generation and more.

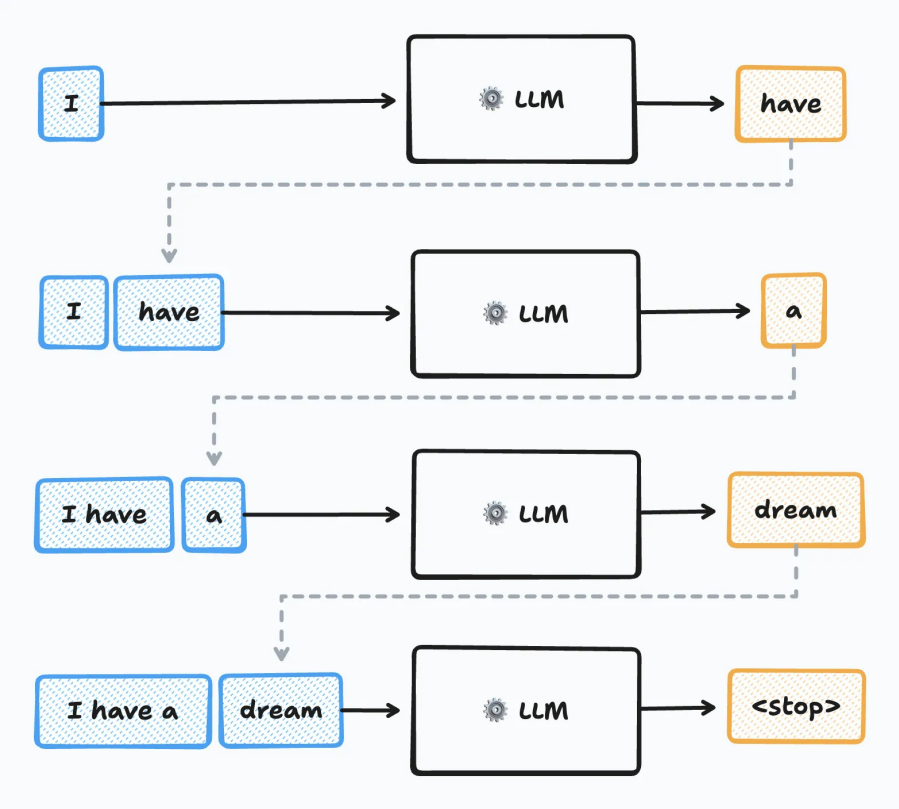


Figure 1.1: Adopted from llm

1.1 Stages in the Model Creation Pipeline

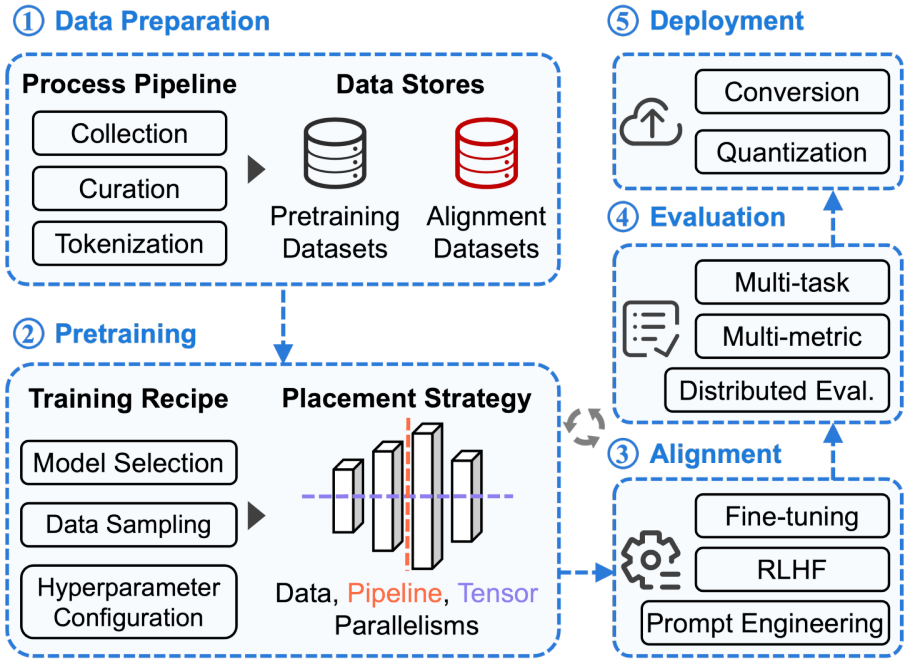


Figure 1.2: Adopted from USE

- **Data Preparation:** The initial stage involves gathering and preprocessing the training data, which can be categorized into two parts: (1) pretraining data, consisting of extensive unlabeled corpora obtained from public or private sources and curated through processes like detoxification and deduplication; (2) alignment data, comprising a smaller set of high-quality labeled corpora used to align the model with specific tasks. This data is typically acquired through expensive human annotation or labeling. Besides, all the data must be tokenized to ensure compatibility with the model's input.

- **Pretraining:** It involves self-supervised training on large-scale curated data, demanding a majority of resources within the overall development workflow. Training LLMs efficiently at scale necessitates various system innovations, such as state-sharding optimizers, meticulous model placement using data, pipeline, and tensor parallelisms.

- **Alignment:** This stage aims to adapt LLMs with user intent on a wide range of downstream tasks. Two primary aligning paradigms are commonly used: (1) prompt engineering, specifying prompts (i.e., inputs) without modifying model parameters. For example, in text summarization, appending a prompt "TL; DR" to the input article can improve model performance; (2) fine-tuning, updating model parameters on a task-specific dataset to improve performance in a particular domain.

- **Evaluation:** Given the vast application scenarios of LLM, it may be inaccurate to assess model quality solely based on a single metric like training loss. There are numerous factors to consider, such as accuracy, fairness, and toxicity. Consequently, it is crucial to account for a diverse set of criteria and measure performance across multiple tasks. Furthermore, regular evaluation is essential during the pretraining stage to provide timely feedback on model quality.

- **Deployment:** To meet the strict cost and latency constraints of LLM applications, several advanced techniques have been developed to achieve efficient model serving, including quantization, distillation, CUDA kernel optimization, model parallelism and memory management.

1.2 Transformer Architecture

1.2.1 Tokenization

Tokenization is the process of converting raw text into a sequence of tokens that the model can process. Instead of working directly with characters or words, modern language models often rely on subword tokenization methods such as Byte Pair Encoding (BPE), WordPiece, or SentencePiece. These approaches break words into smaller, more frequent units, balancing vocabulary size and representation power. For example, the word "unhappiness" may be split into "un", "happiness", while rare words are decomposed into character-level units. Tokenization ensures consistent representation of unseen or rare words and significantly reduces out-of-vocabulary issues.

1.2.2 Embedding and Attention

Once tokenized, tokens are mapped into continuous vector representations called embeddings. These embeddings capture semantic and syntactic relationships among words, such that similar words are close in the embedding space. Positional encodings are added to embeddings to preserve word order information, since attention mechanisms are permutation invariant by default.

Attention is the core mechanism of transformer models. It allows the model to selectively focus on different parts of the input sequence when generating or predicting tokens. Given a query, attention computes weighted similarities with key-value pairs, enabling contextualized representations that consider both local and long-range dependencies.

1.2.3 Multi-Head Masked Attention

Masked attention is used in autoregressive language models to prevent the model from "looking ahead" at future tokens during training. This masking enforces causality by ensuring that each token can only attend to itself and preceding tokens. Multi-head attention extends this concept by computing multiple attention functions in parallel. Each head captures different aspects of relationships—such as syntactic dependencies, semantic similarity, or positional cues. The outputs of all heads are concatenated and linearly transformed, providing a richer, more expressive representation of the input.

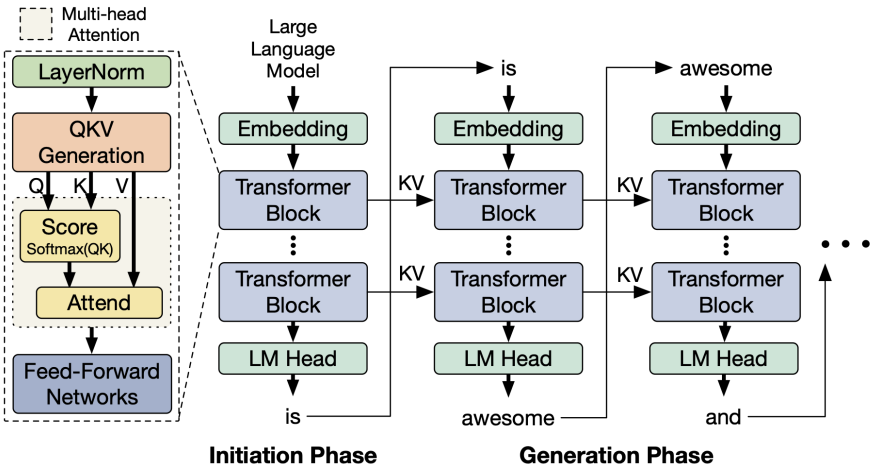


Figure 1.3: Architecture of a Large Language Model (3)

1.3 Transformer Based Large Langauge Models

The task of language modeling is to model the probability of a list of tokens $(x_1, \ldots, x_n)$. Since language has a natural sequential ordering, it is common to factorize the joint probability over the whole sequence as the product of conditional probabilities (a.k.a. autoregressive decomposition):

$$P(x) = P(x_1) \cdot P(x_2|x_1) \ldots P(x_n|x_1, \ldots x_{n-1})$$

Transformers have become the de facto standard architecture for modeling the probability above at a large scale. The most important component of a Transformer-based language model is its self-attention layers. For an input hidden state sequence $(x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$, a self-attention layer first applies linear transformations on each position i to get the query, key, and value vectors:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

Then, the self-attention layer computes the attention score a_{ij} by multiplying the query vector at one position with all the key vectors before it and compute the output o_i as the weighted average over the value vectors:

$$a_{ij} = \frac{exp(q_i^T k_j / \sqrt{d})}{\sum_{t=1}^{i} exp(q_i^T k_t / \sqrt{d})}, \quad o_i = \sum_{j=1}^{i} a_{ij} v_j$$

Besides the above computation, all other components in the Transformer model, including the embedding layer, feed-forward layer, layer normalization, residual connection, output logit computation, and the query, key, and value transformation, are all applied independently position-wise in a form of $y_i = f(x_i)$.

- **Query (Q):** A representation of the current token, used to score its relevance against all past tokens. During decoding, we only compute the query for the token currently being processed.
- **Key (K):** Keys act like labels for tokens in the sequence. They are used to match queries and determine which past tokens are relevant.

- **Value (V):** Values are the actual token representations. Once relevance scores are computed (using Q and K), the corresponding values are aggregated to produce the output representation.

1.3.1 Autoregressive Generation

A request to an LLM service provides a list of input prompt tokens $(x_1, \ldots, x_n)$,and the LLM service generates a list of output tokens $(x_{n+1}, \ldots, x_{n+T})$. Due to the decomposition, the LLM can only sample and generate new tokens one by one, and the generation process of each new token depends on all the previous tokens in that sequence, specifically their key and value vectors. In this sequential generation process, the key and value vectors of existing tokens are often cached for generating future tokens, known as KV cache. Note that the KV cache of one token depends on all its previous tokens. This means that the KV cache of the same token appearing at different positions in a sequence will be different.

Given a request prompt, the generation computation in the LLM service can be decomposed into two phases:

The prompt phase takes the whole user prompt $(x_1, \ldots, x_n)$ as input and computes the probability of the first new token $P(x_{n+1}|x_1, \ldots, x_n)$. During this process, also generates the key vectors $k_1, \ldots, k_n$ and value vectors $v_1, \ldots, v_n$. Since prompt tokens $x_1, \ldots, x_n$ are all known, the computation of the prompt phase can be parallelized using matrix- matrix multiplication operations. Therefore, this phase can efficiently use the parallelism inherent in GPUs.

The autoregressive generation phase generates the remaining new tokens sequentially. At iteration t the model takes one token x_{n+t} as input and computes the probability $P(x_{n+t+1}|x_1, \ldots, x_{n+t})$ with the key vectors $k_1, \ldots, k_{n+t}$ and value vectors $v_1, \ldots, v_{n+t}$. Note that the key and value vectors at positions 1 to $n + t - 1$ are cached at previous iterations, only the new key and value vector k_{n+t} and v_{n+t} are computed at this iteration. This phase completes either when the sequence reaches a maximum length (specified by users or limited by LLMs) or when an end-of-sequence (<eos>) token is emitted. The computation at different iterations cannot be parallelized due to the data dependency and often uses matrix-vector multiplication, which is less efficient. As a result, this phase severely underutilizes GPU computation and becomes memory-bound, being responsible for most portion of the latency of a single request.

1.4 LLM Inferencing

Large Language Model (LLM) inferencing refers to the process of using a trained model to generate predictions (such as text completions, answers, translations, etc.) given some input. Unlike training, where parameters are updated, inferencing only runs the trained model forward to produce outputs.

1.4.1 Forward Pass

- The forward pass is the core computation step in inferencing.
- Input tokens (converted to embeddings) are passed layer by layer through the transformer blocks:
  - Embedding → Attention → Feedforward → Normalization (repeated across layers).

- Each forward pass produces *logits*: a vector of unnormalized scores for the next token in the vocabulary.

1.4.2 Token Probability Distribution

- The logits are passed through a softmax to get probabilities.
- These probabilities define the model's belief about which token should come next.
- Different decoding strategies may be applied:
  - **Greedy decoding**: pick the token with maximum probability.
  - **Sampling**: sample a token according to the probability distribution.
  - **Top-k / Top-p sampling**: restrict sampling to the most probable tokens.
  - **Beam search**: keep multiple hypotheses alive and expand them.

1.4.3 Generation Loop

- Inferencing is auto-regressive:
  1. Generate one token.
  2. Append it to the context.
  3. Feed back into the model.
- Repeat until a stopping criterion (e.g., end-of-sequence token or maximum length) is reached.

1.4.4 Modes of Inference

- **Forward only** (logits for a given input): used in classification, scoring, and embeddings.
- **Generate** (auto-regressive token loop): used in chatbots, summarization, and translation.
- **Hybrid tasks**: e.g., retrieval-augmented generation (RAG), where inference integrates external knowledge.

1.5 Inference Engines

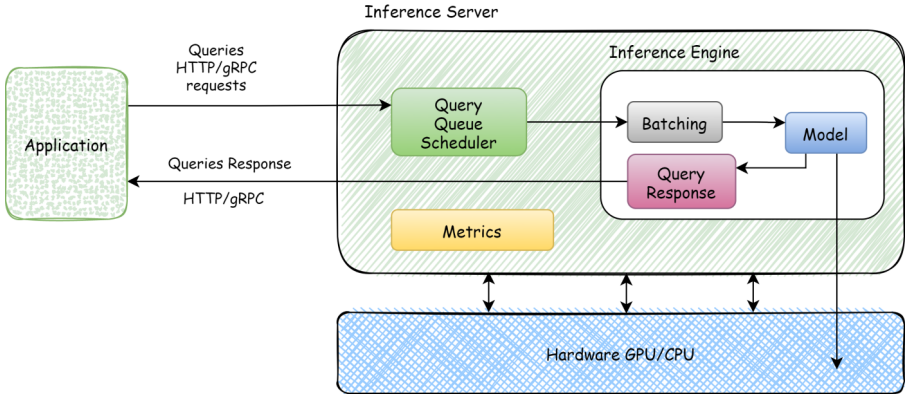


Figure 1.4: Architecture of an Inference Engine (3)

An **inference engine** is the software and systems layer that manages the execution of large language model (LLM) inference on hardware resources. While the model architecture and parameters define the computation, inference engines ensure that requests are executed efficiently, reliably, and at scale.

- **Batching:** Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead. Efficient batching is crucial for achieving high throughput in production environments.
- **Streaming:** Outputs are generated token-by-token and streamed back to the user as they are produced, reducing latency for interactive applications such as chatbots.
- **Fault Management:** Inference engines must be resilient to hardware or software failures. This includes checkpointing, task retries, and robust scheduling.
- **Error Handling:** Engines provide mechanisms for handling issues such as invalid inputs, out-of-memory errors, or timeouts without disrupting other active requests.
- **Parallelism (Multi-GPU):** Large models often cannot fit on a single GPU. Inference engines handle model parallelism and pipeline parallelism to distribute computation across multiple GPUs or nodes, while minimizing communication overhead.

1.5.1 Optimizations in Inference Engines

- **Memory Storage Optimizations:** Inference engines carefully manage GPU memory by caching key-value pairs (KV cache), reusing memory buffers, and applying quantization or weight-sharing techniques to reduce footprint.
- **Offloading:** Portions of the model state or intermediate activations can be offloaded to CPU or other storage when not actively needed on the GPU. This allows inference of models larger than GPU memory capacity.

1.6 Prefill vs Decode

LLM inference consists of two phases:

- **Prefill:** The input/prompt is ingested in parallel. All tokens are processed together to build the key-value (KV) cache, enabling the generation of the first output token.
- **Decode:** Subsequent tokens are generated auto-regressively. Each new token is computed sequentially using the KV cache from earlier steps.

In short, *prefill is parallel and one-time*, while *decode is sequential and repeated.*

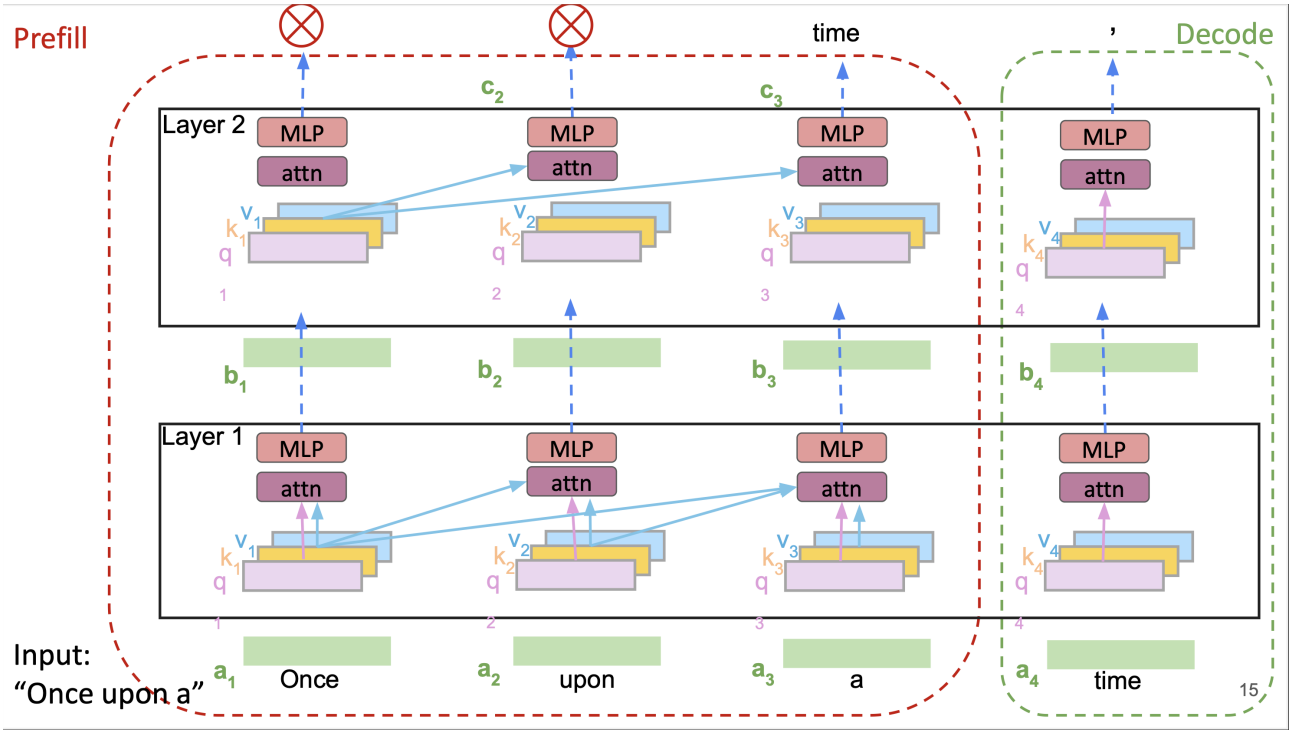


Figure 1.5: Prefill vs Decode

1.7 Metrics

The performance of LLM inference is typically measured using two key metrics:

1.7.1 Throughput

- Defined as the number of tokens processed per second.
- Reflects system efficiency and GPU utilization across multiple requests.

1.7.2 Latency

- Measures the responsiveness of the system.
- **Time to first token (TTFT):** Corresponds to the prefill phase, i.e., the delay before the first output token is produced.

- **Time between tokens:** Corresponds to the decode phase, i.e., the per-token delay during sequential generation.

Chapter 2

vLLM

vLLM an LLM serving system that achieves (1) near-zero waste in KV cache memory and (2) flexible sharing of KV cache within and across requests to further reduce memory usage. It also proposes PagedAttention, an attention algorithm inspired by the classical virtual memory and paging techniques in operating systems.

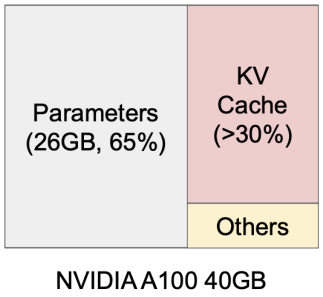


Figure 2.1: Memory layout when serving an LLM with 13B parameters on NVIDIA A100

Approximately 65% of the memory is allocated for the model weights, which remain static during serving. Close to 30% of the memory is used to store the dynamic states of the requests. For Transformers, these states consist of the key and value tensors associated with the attention mechanism. The remaining small percentage of memory is used for other data, including activations - the ephemeral tensors created when evaluating the LLM. Since the model weights are constant and the activations only occupy a small fraction of the GPU memory, the way the KV cache is managed is critical in determining the maximum batch size.

Unlike the tensors in the traditional deep learning workloads, the KV cache has unique characteristics: it dynamically grows and shrinks over time as the model generates new tokens, and its lifetime and length are not known a priori. These characteristics make the existing systems' approach significantly inefficient in two ways: First, the existing systems suffer from internal and external memory fragmentation. To store the KV cache of a request in contiguous space, they pre-allocate a contiguous chunk of memory with the request's maximum length (e.g., 2048 tokens). This can result in severe internal fragmentation, since the request's actual length can be much shorter than its maximum length. Besides, external memory fragmentation can also be significant, since the preallocated size can be different for each request. Second, the existing systems cannot exploit the opportunities for memory sharing. LLM services often use advanced decoding algorithms, such as parallel sampling and beam search, that generate multiple outputs per request. In these scenarios, the request consists of multiple sequences that

can partially share their KV cache. However, memory sharing is not possible in the existing systems because the KV cache of the sequences is stored in separate contiguous spaces.

2.1 Memory Challenges

- **Large KV Cache:** The KV Cache size grows quickly with the number of requests. As an example, for the 13B parameter OPT model, the KV cache of a single token demands 800KB of space, calculated as 2 (key and value vectors)$\times$ 5120 (hidden state size) $\times$ 40 (number of layers) $\times$ 2 (bytes per FP16). Since OPT can generate sequences up to 2048 tokens, the memory required to store the KV cache of one request can be as much as 1.6 GB.
- **Complex decoding algorithms:** LLM services offer a range of decoding algorithms for users to select from, each with varying implications for memory management complexity.
- **Scheduling for unknown input & output lengths:** The requests to an LLM service exhibit variability in their input and output lengths. This requires the memory management system to accommodate a wide range of prompt lengths. In addition, as the output length of a request grows at decoding, the memory required for its KV cache also expands and may exhaust available memory for incoming requests or ongoing generation for existing prompts.

2.2 vLLM

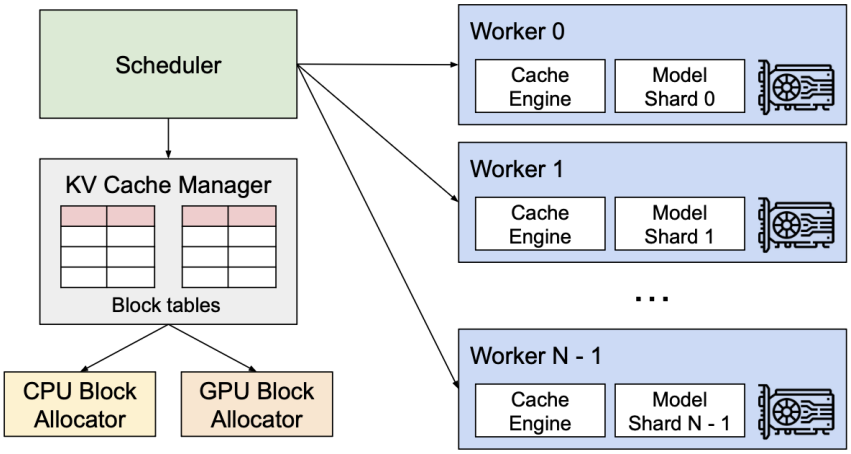


Figure 2.2: vLLM system overview.

vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers. The KV cache manager effectively manages the KV cache in a paged fashion, enabled by PagedAttention. Specifically, the KV cache manager manages the physical KV cache memory on the GPU workers through the instructions sent by the centralized scheduler.

2.2.1 PagedAttention

PagedAttention partitions the KV cache of each sequence into KV blocks. Each block contains the key and value vectors for a fixed number of tokens. During the attention computation,

the PagedAttention kernel identifies and fetches different KV blocks separately. In summary, the PagedAttention algorithm allows the KV blocks to be stored in non-contiguous physical memory, which enables more flexible paged memory management in vLLM.

2.2.2 KV Cache Manager

A request's KV cache is represented as a series of logical KV blocks, filled from left to right as new tokens and their KV cache are generated. The last KV block's unfilled positions are reserved for future generations. On GPU workers, a block engine allocates a contiguous chunk of GPU DRAM and divides it into physical KV blocks (this is also done on CPU RAM for swapping). The KV block manager also maintains block tables—the mapping between logical and physical KV blocks of each request. Each block table entry records the corresponding physical blocks of a logical block and the number of filled positions. Separating logical and physical KV blocks allows vLLM to dynamically grow the KV cache memory without reserving it for all positions in advance, which eliminates most memory waste in existing systems.

2.2.3 Parallel Sampling

In parallel sampling, one request includes multiple samples sharing the same input prompt, allowing the KV cache of the prompt to be shared as well. Since both outputs share the same prompt, space is reserved only for one copy of the prompt's state at the prompt phase; the logical blocks for the prompts of both sequences are mapped to the same physical blocks. Since a single physical block can be mapped to multiple logical blocks, a reference count is introduced for each physical block. At the generation phase, the two outputs sample different output tokens and need separate storage for KV cache. vLLM implements a copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences. Unlike parallel decoding, beam search facilities sharing not only the initial prompt blocks but also other blocks across different candidates, and the sharing patterns dynamically change as the decoding process advances.

2.2.4 Scheduling and Preemption

Typically, eviction policies use heuristics to predict which block will be accessed furthest in the future and evict that block. Since in our case it's known that all blocks of a sequence are accessed together, we implement an all-or-nothing eviction policy, i.e., either evict all or none of the blocks of a sequence. Furthermore, multiple sequences within one request (e.g., beam candidates in one beam search request) are gang-scheduled as a sequence group. The sequences within one sequence group are always preempted or rescheduled together due to potential memory sharing across those sequences. There are two techniques to recover an evicted block:

Swapping

In vLLM's case, evicted blocks are copiedto the CPU memory. vLLM includes a CPU block allocator to manage the physical blocks swapped to CPU RAM. When vLLM exhausts free physical blocks for new tokens, it selects a set of sequences to evict and transfer their KV cache to the CPU. Once it preempts a sequence and evicts its blocks, vLLM stops accepting new requests until all preempted sequences are completed. Once a request completes, its blocks are freed from memory, and the blocks of a preempted sequence are brought back in to continue the processing of that sequence. Note that with this design, the number of blocks swapped to

the CPU RAM never exceeds the number of total physical blocks in the GPU RAM, so the swap space on the CPU RAM is bounded by the GPU memory allocated for the KV cache.

Recomputation

In this case, we simply recompute the KV cache when the preempted sequences are rescheduled. The performances of swapping and recomputation depend on the bandwidth between CPU RAM and GPU memory and the computation power of the GPU.

2.3 Automatic Prefix Caching in vLLM

Prefix caching kv-cache blocks is a popular optimization in LLM inference to avoid redundant prompt computations. The core idea is simple - we cache the kv-cache blocks of processed requests, and reuse these blocks when a new request comes in with the same prefix as previous requests. Since prefix caching is almost a free lunch and won't change model outputs, it has been widely used by many public endpoints (e.g., OpenAI, Anthropic, etc) and most open source LLM inference frameworks (e.g., SGLang).

We can build the block hash of hash(tuple[components]), where components are:

- Parent hash value: The hash value of the parent hash block.
- Block tokens: A tuple of tokens in this block. The reason to include the exact tokens is to reduce potential hash value collision.
- Extra hashes: Other values required to make this block unique, such as LoRA IDs, multi-modality input hashes, and cache salts to isolate caches in multi-tenant environments.

There are two design points to highlight:

- All KVCacheBlocks are allocated when initializing the KV cache manager to be a block pool. This avoids Python object creation overheads and can easily track all blocks all the time.
- Doubly linked list pointers are introduced directly in the KVCacheBlock, so that a free queue can be constructed directly.

2.3.1 Components

- **Block Pool:** A list of `KVCacheBlock` instances.
- **Free Block Queue:** Stores only the pointers of head and tail blocks for efficient manipulation.
- **Cache Blocks:** Mapping from hash keys to block IDs.
- **Request Blocks:** Mapping from request IDs to allocated block IDs.

2.3.2 Operations

Block Allocation

When the scheduler schedules a new request:

1. Calls `kv_cache_manager.get_computed_blocks()` to obtain the sequence of precomputed blocks using prompt token hashes.
2. Calls `kv_cache_manager.allocate_slots()`, which:
   (a) Computes the number of required blocks and exits if insufficient blocks are available.
   (b) "Touches" the computed blocks, incrementing their reference count and removing them from the free queue to avoid eviction.
   (c) Allocates new blocks from the free queue head. If the head block is cached, it is evicted.
   (d) If a block becomes full, it is immediately added to the cache for reuse within the same batch.

When scheduling a running request:

1. The scheduler again calls `allocate_slots()`.
2. New blocks are allocated from the free queue head, evicting cached blocks if necessary.
3. Token IDs are appended to both existing and new blocks. Full blocks are added to the cache.

Free Operation

When a request completes:

- All blocks with reference count = 0 are freed.
- Freed blocks are appended to the *tail* of the free queue in reverse order, since later blocks hash more tokens and are less likely to be reused.

Eviction (LRU Policy)

When the least recently used (LRU) block at the head of the free queue is cached:

1. Pop the block from the head of the queue.
2. Remove the block ID from the cache and delete its hash entry.

Chapter 3

SGLang

SGLANG is a system for efficient execution of complex language model programs. SGLang consists of a frontend language and a runtime. The frontend simplifies programming with primitives for generation and parallelism control. The runtime accelerates execution with novel optimizations like RadixAttention for KV cache reuse and compressed finite state machines for faster structured output decoding.

3.1 LM Programs

The emergence of few-shot learning, self-consistency, skeleton-of-thought, tree-of-thought and various other patterns signifies a shift in our interaction with LLMs, moving from simple chatting to a more sophisticated form of programmatic usage of LLMs, which means using a program to schedule and control the generation processes of LLMs. These programs are referred to as LM (Language Model) programs. There are two common properties of LM programs:

- LM programs typically contain multiple LLM calls interspersed with control flow. This is needed to complete complex tasks and improve overall quality.
- LM programs receive structured inputs and produce structured outputs. This is needed to enable the composition of LM programs and to integrate LM programs into existing software systems.

3.2 Programming Model

Language primitives: SGLang provides primitives for controlling prompt state, generation, and parallelism. Here are the primitives: "gen" calls a model to generate and stores the results in a variable with the name specified in its first argument. It supports a "regex" argument to constrain the output to follow a grammar defined by a regular expression (e.g., a JSON schema). "select" calls a model to choose the highest probability option from a list. The operator "+=" or "extend" appends a string to the prompt. The operator "[variable_name]" fetches the results of a generation. "fork" creates parallel forks of the prompt state. "join" rejoins the prompt state. "image" and "video" take in image and video inputs.

An example:

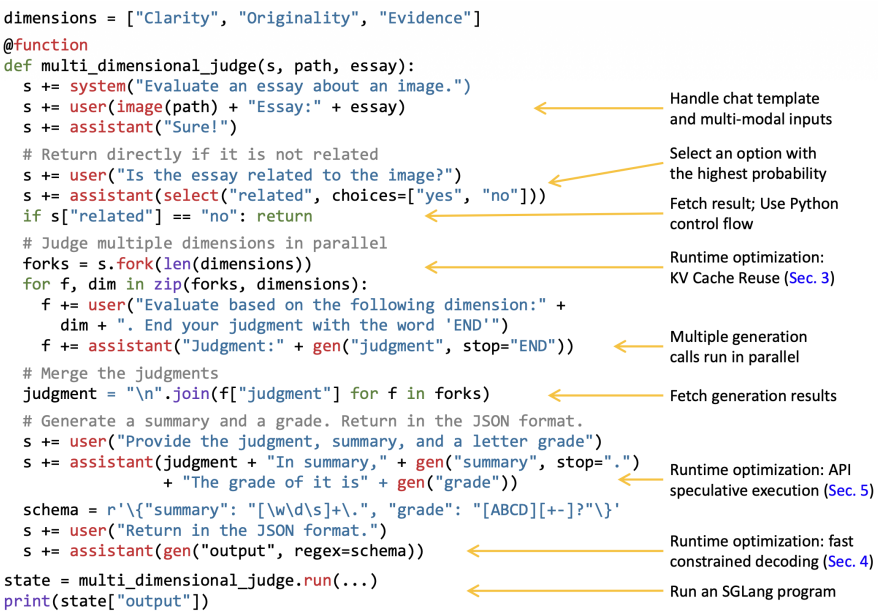


Figure 3.1: SGLANG Frontend Language Model

3.3 RadixAttention

A radix tree is a data structure that serves as a space-efficient alternative to a classical trie (prefix tree). Unlike typical trees, the edges of a radix tree can be labeled not just with single elements but also with sequences of elements of varying lengths, significantly enhancing efficiency. In SGLANG, a radix tree is utilized to manage a mapping between sequences of tokens, and their corresponding KV cache tensors. These KV cache tensors are stored in a non-contiguous, paged layout, where the size of each page is equivalent to one token. Because GPU memory is quickly filled by the KV cahce, a simple LRU eviction policy is introduced that evicts the least recently used leaf first. By evicting leaves first, we enable the re-use of their common ancestors until those ancestors become leaves and are also evicted.

In the continuous batching setting, we cannot evict nodes used by the currently running batch. Each node maintains a reference counter indicating how many running requests are using it. A node is evictable if its reference counter is zero.

The cache hit rate is defined as $\frac{\texttt{number of cached prompt tokens}}{\texttt{number of prompt tokens}}$. If the request scheduler frequently switches between different, unrelated requests, it can lead to cache thrashing and a low hit rate.

In the batch-processing setting the requests are sorted by matched prefix length and prioritize requests with longer matched prefixes instead of using a first-come, first-served schedule.

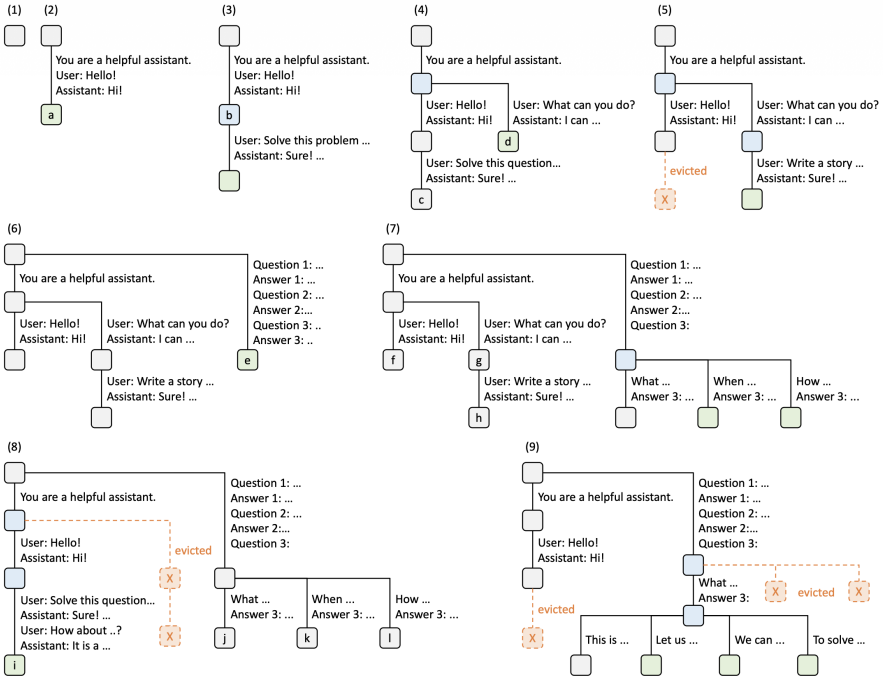


Figure 3.2: SGLANG Radix Tree Example

3.4 Compressed Finite State Machine

In LM programs, users often want to constrain the model's output to follow specific formats, such as JSON schemas. Existing systems support this by converting a regular expression into a finite state machine (FSM). During decoding, they maintain the current FSM state, retrieve allowed tokens from the next states, and set the probability of invalid tokens to zero, decoding token by token. They can only decode one token at a time because the lack of integration between the FSM and the model runner in existing systems prevents multi-token processing, resulting in slow decoding. SGLang overcomes this limitation by creating a fast constrained decoding runtime with a compressed FSM. This runtime analyzes the FSM and compresses adjacent singular-transition edges in the FSM into single edges.

Chapter 4

FastTree

4.1 Introduction

Existing LLM serving systems use a radix tree to organize the global key-value (KV) cache, facilitating cache reuse across different queries and thus reducing unnecessary memory use. Despite this, these systems still rely on conventional computation patterns for attention operations, resulting in redundant memory loads and GPU tensor core underutilization.

FastTree introduces GPU kernels tailored for efficiently processing queries that share contexts through the radix tree.

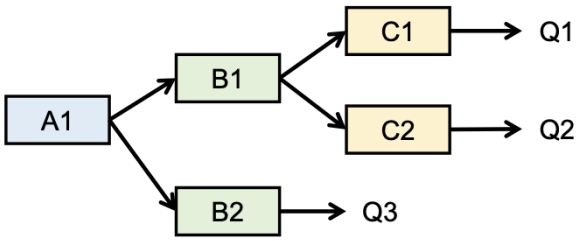


Figure 4.1: KV cache sharing through Radix Tree

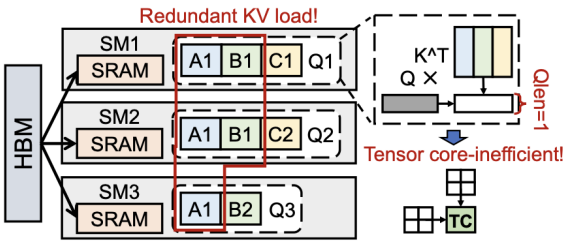


Figure 4.2: Computation pattern of existing systems

However, although optimizing the memory layouts with a radix tree, existing systems still utilize the conventional attention kernels which concatenate the KV cache for each query and dispatch its computation into an individual GPU thread block. It requires redundantly loading the shared KV cache from slow global memory for every query and cannot meet the minimum shape requirements for tensor cores without padding.

With a given radix tree of the KV cache at runtime, we can generate enormous grouping plans by changing the shared contexts used for query aggregation, as the different queries can

share varied prefixes. Moreover, the plan selection can significantly affect the execution performance of the attention kernel, as different plans can introduce distinct overheads, including wasted shared memory and computation due to tile padding and IO cost due to increased intermediate results.

4.2 Tree Structure Adaptive Runtime Optimization

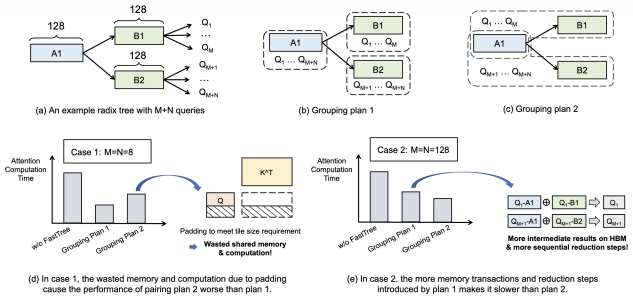


Figure 4.3: Illustration of two context-queries grouping plans

Different grouping plans change the computation-batching and memory-sharing patterns, which affects the performance of the final attention kernel significantly. Moreover, even with the same tree topology, the performance of the same plan can vary in different cases. For example, in the case shown in (d), the queries associated with B1 and B2 are both 8, while the kernel requires a tile size of 16 to utilize the memory and computing resources fully. Hence, for plan 2, the GPU kernel has to rely on input padding to meet the hardware requirements for all groups, while plan 1 can avoid this for the long-context A1 group. As a result, plan 2 introduces much more wasted computation, making its performance worse than plan 1. In contrast, in another case shown in (e), where M=N=128, both plans can fully utilize the shared memory and tensor cores. However, as plan 1 separates A1 from B1 and B2, there will be more intermediate results stored on the slow global memory and requires more sequential reduction steps in the subsequent GPU kernel.

Formally, given a KV cache tree $T = (V, E)$ at runtime, where V represents the set of vertices and E the set of edges, the objective is to determine an assignment function $f : E \to \{0, 1\}$ that minimizes the latency of GPU kernel execution. Specifically, we aim to solve:

$$min_f Latency(Groups(VT(T, f(E))))$$

s.t. $f(e) \in \{0, 1\}, \forall e \in E$ where $VT(T, f(E))$ transforms the original tree T into a virtual tree based on the binary edge assignment f(E).

Algorithm 1 Greedy Binary Edge Assignment

Input: Radix tree $T = (V, E)$

Initialize A as the number of all associated queries of each node in V
Initialize L as the context length of each node
for node v in BFS(V) **do**
  $nQ_{curr} = A[v]$ ▷ current #aggregated queries
  $len_v = L[v]$ ▷ accumulated context length
  for l in leaves(v) **do**
    $nQ_l = A[l]$, $len_l = L[l]$
    $C_0 = \text{SplitKVCost}(nQ_{curr}, nQ_l, len_l, len_v)$
    $C_1 = \text{SplitQCost}(nQ_{curr}, nQ_l, len_l, len_v)$
    if $C_0 \geq C_1$ **then**
      Assign 1 to edge $v \to l$
      $nQ_{curr} = nQ_{curr} - nQ_l$
      $L[l] = len_l + len_v$
    else
      Assign 0 to edge $v \to l$
    end if
  end for
end for

Figure 4.4: Greedy Algorithm

Cost due to padding

Padding introduces two primary sources of performance overhead. First, it leads to inefficient utilization of shared memory, which could otherwise be used to cache additional aggregated queries for improved data reuse and reduced global memory access. Second, padding results in redundant computations that do not contribute to the final output. To estimate the overhead due to padding at the query dimension, we define its cost function by combining the increased memory loading and floating-point operations:

$$C_{P,q}(n_Q, len) = \text{Pad}(TS_q, n_Q) \cdot len \cdot d \tag{4.1}$$

where n_Q represents the query number, len is the context length, TS_q is the tile size, and d is the head dimension. Similarly, we have the cost function for padding at the context dimension:

$$C_{P,c}(n_Q, len) = n_Q \cdot \text{Pad}(TS_c, \min(len, TS_c)) \cdot d \tag{4.2}$$

The overall padding cost of a virtual node is then defined as:

$$C_P(n_Q, len) = \alpha \cdot C_{P,q}(n_Q, len) + \beta \cdot C_{P,kv}(n_Q, len) \tag{4.3}$$

where α and β are two empirical coefficients.

Cost due to intermediate results

Besides padding, splitting the KV matrices at the context dimension (i.e., assigning 0 to the edge) can introduce more intermediate results and cause additional overhead. This is because the attention computation requires reducing all elements across the context dimension. After context splitting, different groups are processed by separate thread blocks, so the intermediate partial reduction results should be stored in global memory, which causes extra memory transaction overhead and brings more sequential reduction steps in the next stage to compute the final results.

$$\text{SplitKV_Cost}_R = \gamma \cdot n_Q^l \cdot d \tag{4.4}$$

$$\text{SplitKV_Cost}_P = C_P(n_Q^{curr}, len_v) + C_P(n_Q^l, len_l) \tag{4.5}$$

$$\text{SplitQ_Cost} = C_P(n_Q^{curr} - n_Q^l, len_v) + C_P(n_Q^l, len_v + len_l) \tag{4.6}$$

Then, the total cost of assigning a value of 0 is:

$$\text{SplitKV_Cost} = \text{SplitKV_Cost}_P + \text{SplitKV_Cost}_R \tag{4.7}$$

4.3 Efficient Tree Structure Attention Kernel Design

Within the kernel, they perform the attention operation tile-by-tile. For each group, they divide the QKV matrices associated with it into multiple tiles and parallelize the tiles across the query dimension by dispatching the Q matrix tiles into different blocks. In contrast, they sequentially iterate the K and V tiles within a block due to the inter-tile dependency of the outputs across the context dimension. They first copy the QKV tiles from the GPU global memory to the fast on-chip shared memory, and then utilize the online softmax technology to store all partial softmax results in the shared memory.

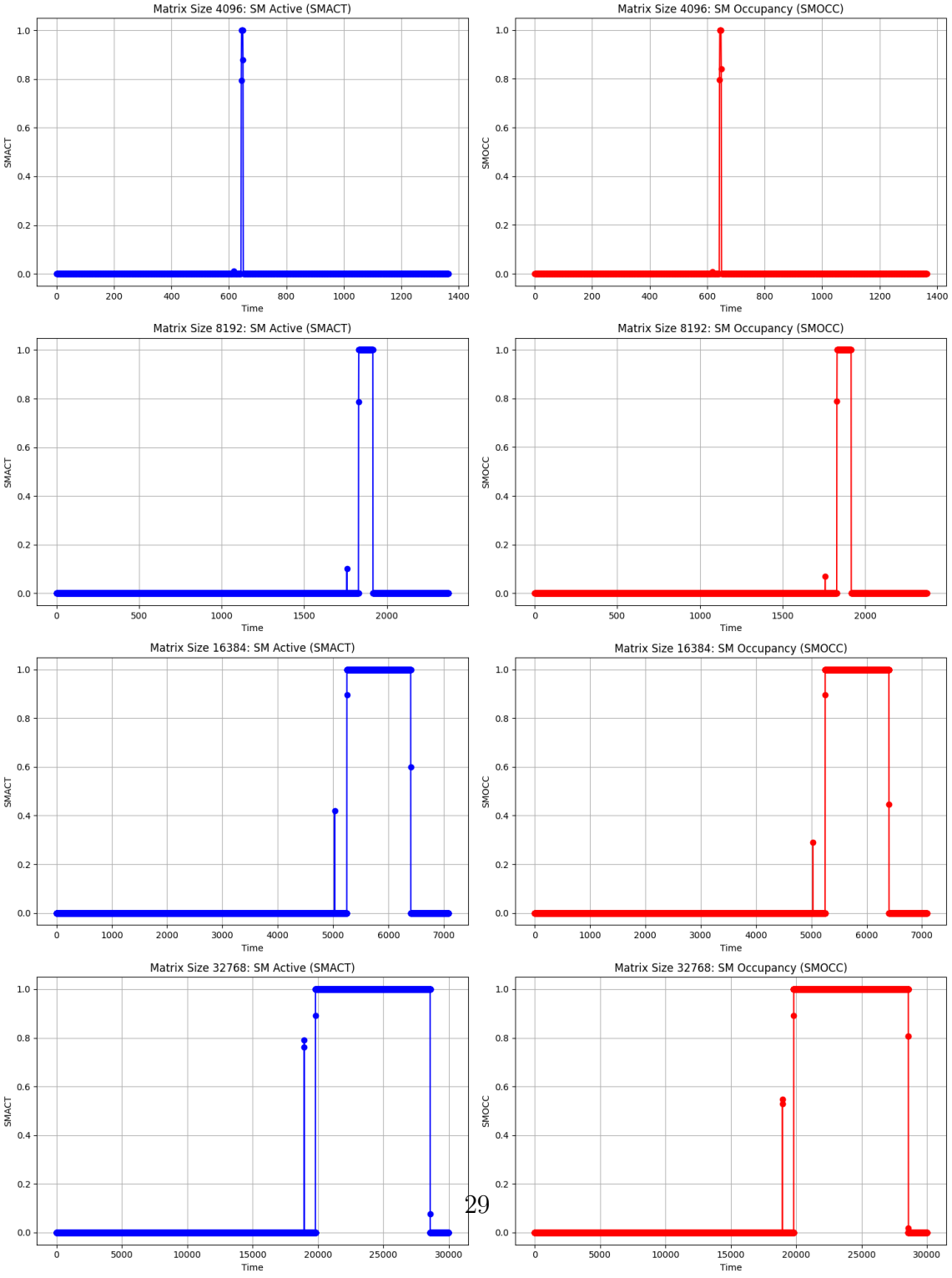
Moreover, by aggregating queries, they can transform the GEMV operation into the GEMM operation, thus enabling the effective usage of tensor cores on the GPU.

Chapter 5

SwiftLLM

Chapter 6

Matrix Multiplication Analysis



Chapter 7

Experiments

7.1 System Prompt in Cache Experiment

The basic application which we want to cater is inferencing in cases where the system prompt is common across multiple queries. And therefore, we want to utilize caching by already filling the KV-Cache with the system prompt, and then putting user queries on the top of it.

7.1.1 Example 1

I picked the 200-word summary of the demand paging lecture as the system prompt to be provided to the LLM. The model which we used was `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (1.1 billion parameters). The system prompt provided was:

> The lecture discusses demand paging, a key concept in virtual memory management. Instead of loading all pages of a process into memory, only the required pages are brought in, while the rest may stay in swap space on disk. This allows processes to use address spaces larger than physical memory and enables efficient memory utilization.
> Each process has a page table, with entries containing a valid bit (indicating whether the page is part of the process) and a present bit (whether it is currently in memory). If valid=0, the process makes an illegal access. If valid=1 but present=0, a page fault occurs. The OS then allocates a memory frame, possibly evicts a victim page using a page replacement policy, updates the page table, and resumes execution.
> Victim pages may be file-backed or anonymous, and may be dirty (modified) or clean. Dirty pages require writing back to disk before eviction. Policies like FIFO and LRU determine which pages to evict; LRU is effective due to locality, while FIFO can suffer from Belady's anomaly.
> Excessive page faults can cause thrashing, where the system spends more time swapping pages than doing useful work. To prevent this, the OS ensures that each process's working set fits in memory.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.339 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

> **Query 1:** What is the difference between a valid bit and a present bit?

> **Response 1:** The difference between a valid bit and a present bit is that a valid bit indicates that a page is part of a process, while a present bit indicates that a page is currently in memory.

Query 2: Why does thrashing occur in demand paging systems?

Response 2: Thrashing occurs in demand paging systems because excessive page faults can cause the system to spend more time swapping pages than doing useful work. This is because demand paging systems allocate memory frames to processes based on their page table entries, which are either valid or present. If a process has a page fault, it must be allocated a memory

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.1 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.94 seconds.

7.1.2 Example 2

Let us take another example, this time with a summary of the semaphores lecture. The system prompt provided was:

The lecture introduces semaphores, a synchronization primitive similar to condition variables, used to coordinate threads. A semaphore maintains a counter, initialized to some value. The down (wait) operation decrements the counter and blocks the thread if it becomes negative, while the up (post) operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations.
Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They are also useful for synchronization between threads, e.g., enforcing execution order (T1 before T2) or ensuring two threads finish one step before starting another. Incorrect usage can cause deadlocks, such as when both threads wait on each other.
The producer-consumer problem is revisited using semaphores. Two semaphores track empty and filled buffer slots, while a binary semaphore (mutex) ensures mutual exclusion on buffer access. Misuse, like holding locks during blocking down calls, can also lead to deadlocks.
Guidelines emphasize careful initialization, ensuring every down has a corresponding up, and using separate semaphores for signaling and locking. The lecture also covers batched processing with semaphores, showing how request threads and a batch processor can coordinate efficiently.
Semaphores thus provide flexible yet tricky synchronization, requiring careful design to avoid deadlocks.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.347 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

Query 1: How does the down operation on a semaphore synchronize threads during execution?

Response 1: A semaphore is a synchronization primitive that maintains a counter, initialized to some value. The down operation decrements the counter and blocks the thread if it becomes negative. The up operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations. Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They

Query 2: Why is an extra binary semaphore (mutex) needed in producer-consumer implementation?

Response 2: An extra binary semaphore (mutex) is needed in producer-consumer implementation because the producer and consumer threads can access the same buffer at the same time. This can lead to deadlocks if the buffer is not properly protected.

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.774 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.957 seconds.

7.1.3 Observations

- As we can see, there is roughly an improvement of 0.2-0.7 seconds for an experiment of 2 queries, which is also intutively clear, because we are just passing over the system prompt once.
- As we increase the size of the system prompt, and the number of tokens we generate, the difference becomes even wider.
- The time taken to feed through the system prompt, and cloning the KV cache values, can be ignored safely because we can do this before we get the actual user queries.
- The first query of the second example didn't get a superior response, however this is independent from our experiment settings, and depends more on the type of model we use, for example increasing the number of parameters (using a larger model) would definitely help.

7.1.4 Analysis over different steps and using different metrics

I now use a profiler to actually figure out at what all places, the most amount of time is being spent:

Table 7.1: Profiler Output

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
aten::matmul	2.67%	56.256ms	30.07%	632.976ms	40.575us	0.000us	0.00%	589.952ms	37.817us	15600
aten::linear	1.16%	24.342ms	30.78%	647.989ms	41.806us	0.000us	0.00%	589.802ms	38.052us	15500
aten::mm	14.46%	304.341ms	22.68%	477.495ms	30.806us	579.871ms	69.41%	589.802ms	38.052us	15500
cutlass kernel (relu bf16)	0.00%	0.000us	0.00%	0.000us	0.000us	234.992ms	28.13%	234.992ms	65.385us	3594
ampere_bf16_128x64_sliced1x2	0.00%	0.000us	0.00%	0.000us	0.000us	124.274ms	14.87%	124.274ms	66.457us	1870
ampere_bf16_128x128_stages_32x1	0.00%	0.000us	0.00%	0.000us	0.000us	94.240ms	11.28%	94.240ms	44.621us	2112
aten::mul	6.55%	137.891ms	10.29%	216.715ms	10.728us	61.543ms	7.37%	61.560ms	3.048us	20200
aten::copy_	3.97%	83.637ms	8.32%	175.218ms	9.679us	50.846ms	6.09%	50.857ms	2.809us	18103
ampere_bf16_128x64_stages_64x3	0.00%	0.000us	0.00%	0.000us	0.000us	44.572ms	5.34%	44.572ms	19.481us	2288
aten::scaled_dot_product_attention	0.84%	17.683ms	5.51%	116.049ms	52.749us	0.000us	0.00%	44.127ms	20.058us	2200
aten::_scaled_dot_product_flash_attention	0.63%	13.224ms	4.67%	98.366ms	44.712us	0.000us	0.00%	44.127ms	20.058us	2200
aten::_flash_attention_forward	1.00%	21.077ms	3.49%	73.410ms	33.368us	44.073ms	5.28%	44.127ms	20.058us	2200
flash_fwd_kernel	0.00%	0.000us	0.00%	0.000us	0.000us	43.833ms	5.25%	43.833ms	20.125us	2178
elementwise_mul_kernel	0.00%	0.000us	0.00%	0.000us	0.000us	37.673ms	4.51%	37.673ms	2.833us	13300
ampere_bf16_64x64_sliced1x4	0.00%	0.000us	0.00%	0.000us	0.000us	34.059ms	4.08%	34.059ms	28.669us	1188
aten::clone	0.87%	18.407ms	6.48%	136.324ms	15.491us	0.000us	0.00%	26.201ms	2.977us	8800
elementwise_copy_kernel	0.00%	0.000us	0.00%	0.000us	0.000us	26.198ms	3.14%	26.198ms	2.977us	8800
aten::add	4.45%	93.743ms	7.17%	150.912ms	11.347us	24.648ms	2.95%	24.657ms	1.854us	13300
aten::to	0.93%	19.568ms	8.33%	175.368ms	12.345us	0.000us	0.00%	24.656ms	1.736us	14205
aten::_to_copy	1.29%	27.125ms	7.40%	155.800ms	16.747us	0.000us	0.00%	24.656ms	2.650us	9303

Now we go over each of the rows, and further analyze the values:

1. `atten::matmul` = Pytorch's high level matrix multiplication. Runs on CPU side, but calls matrix multiply functions (`atten::mm`) on CUDA.

2. `atten::linear` = The linear layer, has large CUDA compute
3. `cutlass kernel (relu bf16)` = Stands for general matrix multiplication, entirely on CUDA
4. `ampere_...` = Specialized Ampere architecture GEMM kernel in BF16 precision for different tile sizes
5. `atten::mul` = Element-wise matrix multiplication, very high call count
6. `aten::copy_` = Copying tensors, very high call count
7. `aten::_flash_attention_forward` = The forward pass of flash attention

7.1.5 Profiling across pipeline stages

Stage	Time (s)
Tokenize system prompt	0.0052
System forward (KV cache fill)	0.3342
Tokenize query	0.0003
Per token (mean / first)	0.0148 / 0.0390
Generation total	0.7517
Decode tokens	0.0002

Table 7.2: Profiling results for various pipeline stages

7.2 Batch of Requests

We will measure the time taken to process a batch of requests, by varying the batch size: Total number of queries: 24

Technique Used	Time (s)
Transformers (batch size = 1)	30.21
Transformers (batch size = 2)	19.51
Transformers (batch size = 3)	18.64
vLLM (no system prompt pre-filling)	0.45
SGLang (fork)	1.03
SGLang (run_batch)	0.40
NEO	0.55

Table 7.3: Measuring the time taken to process a batch of requests using different techniques

7.3 Throughput and Latency Measurements across different techniques

Here is the experiment setup:

- Model: `TinyLlama/TinyLlama-1.1B-Chat-v1.0`

- System Prompt: Roughly 1500 tokens (adopted from Ostep MLFQ schedulung chapter)
- Queries: 100 queries, each of length 10-20 tokens
- Max Output Length: 80 tokens
- Techniques Compared: vLLM (with and without automatic prompt caching), SGLang, swiftLLM

Here is how we generate the load:

Algorithm 1: Workload Generation at Target QPS

Input: LLM model `llm`, list of `prompts`, parameters `sampling_params`, target QPS `qps`
Output: Latencies of requests

```
Initialize empty list latencies ;
Set interval ← 1/qps ;
Set start_time ← current time ;
Set index i ← 0 ;
Initialize empty backlog ;
Set next_time ← current time ;
while i < length(prompts) do
    Set now ← current time ;
    Compute intervals_passed ← ⌊(now − next_time)/interval⌋ + 1 ;
    // By the time we reached here, a few intervals might have passed, so
       we add that many prompts to the backload
    for k = 1 to intervals_passed do
        if i < length(prompts) then
            Append prompts[i] to backlog ;
            Increment i ;
    if backlog is not empty then
        Set batch_start ← current time ;
        Call llm.generate(backlog, sampling_params) ;
        Set batch_latency ← current time − batch_start ;
        // All the requests in the backloag have the same latency because
           they were processed in a batch
        foreach request in backlog do
            Append batch_latency to latencies ;
        Clear backlog ;
    Update next_time ← next_time + intervals_passed × interval ;
    // If we are ahead of the schedule, we sleep until the next request
       time
    Set sleep_time ← max(0, next_time − current time) ;
    sleep(sleep_time) ;
return (latencies) ;
```

We primarily measure the following metrics:

- **Throughput (QPS)**: Total number of requests / Total time taken
- **Average Latency (s)**: Average time taken per request, basically the time a user will have to wait

- **P95 Latency (s)**: 95th percentile latency across all requests, this helps us understand the tail latency, which in turn blocks the next batch of requests

7.4 LLM Request Timelines

7.4.1 Chunked Prefill Enabled

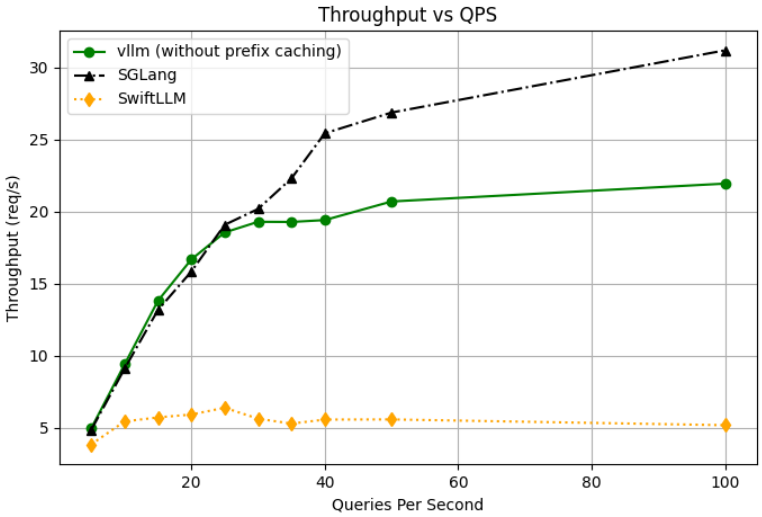


Figure 7.1: Throughput

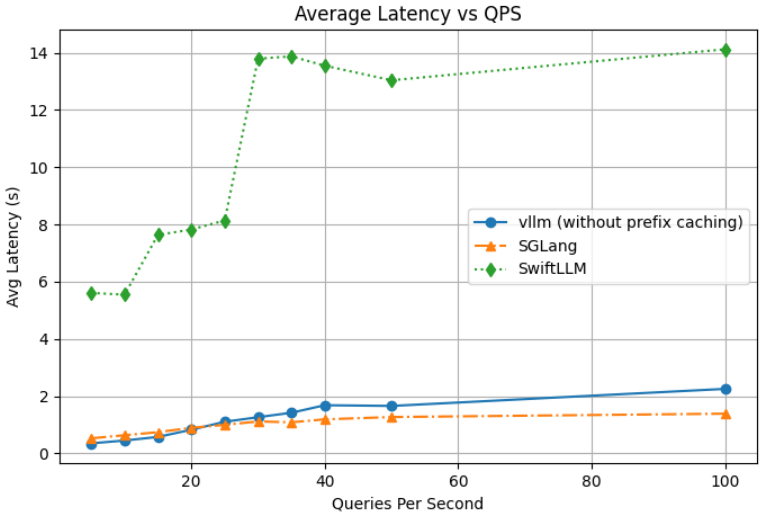


Figure 7.2: Average Latency

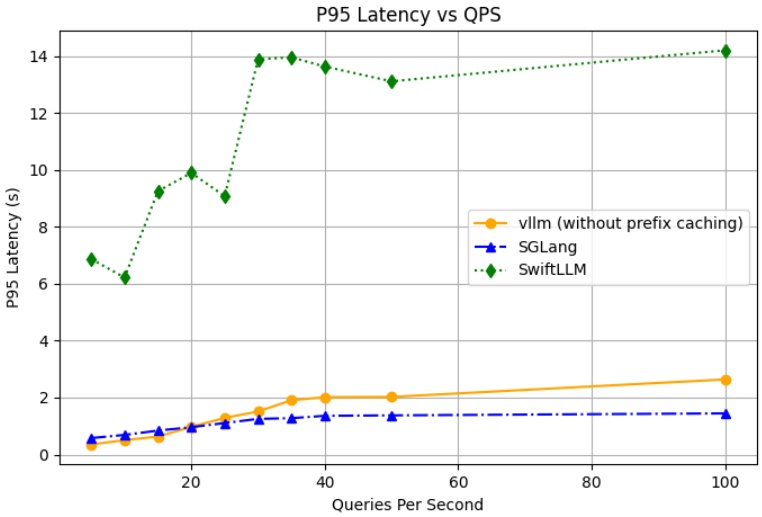


Figure 7.3: P95 Latency

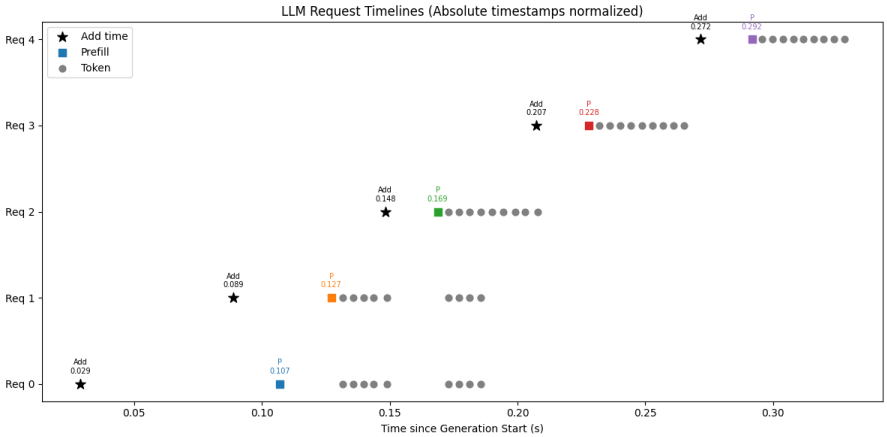


Figure 7.4: 20 Requests per second

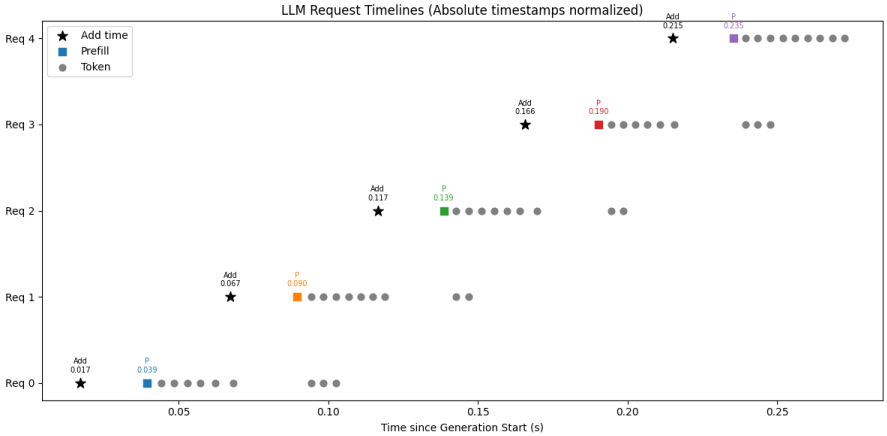


Figure 7.5: 25 Requests per second

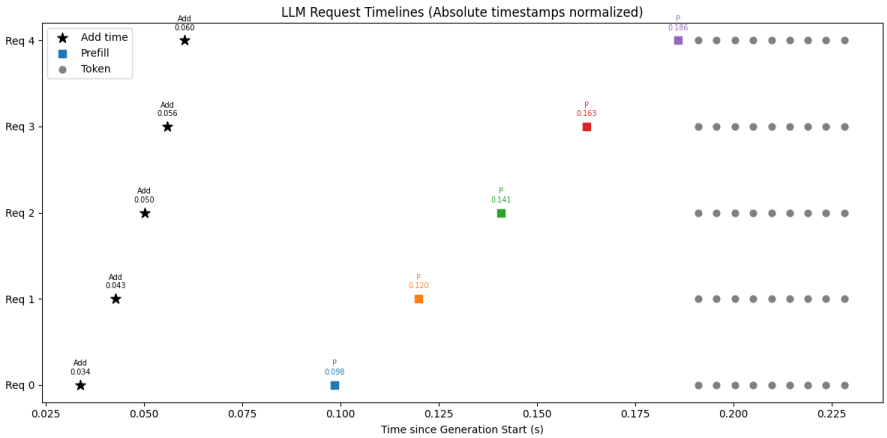


Figure 7.6: All requests at once

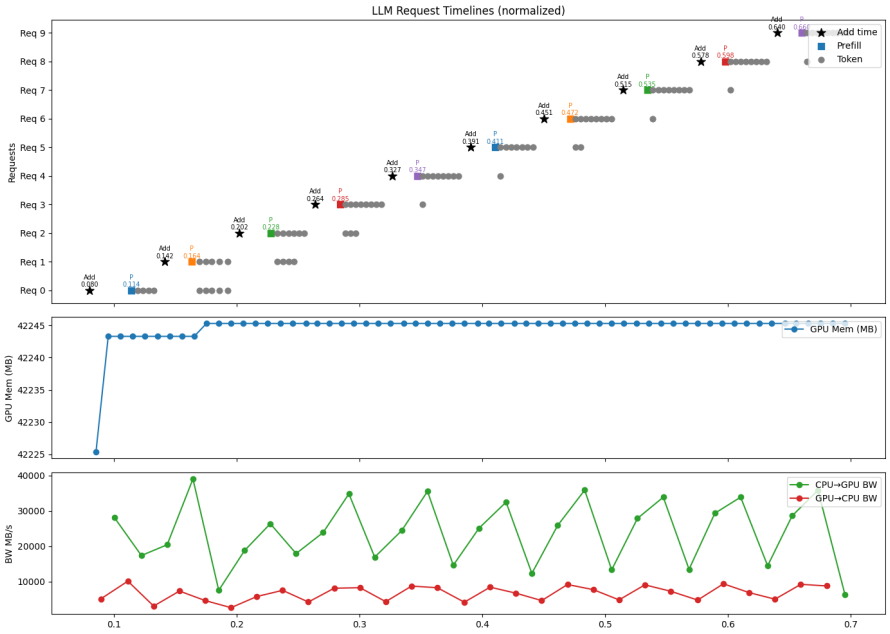


Figure 7.7: Request Timeline with GPU Metrics

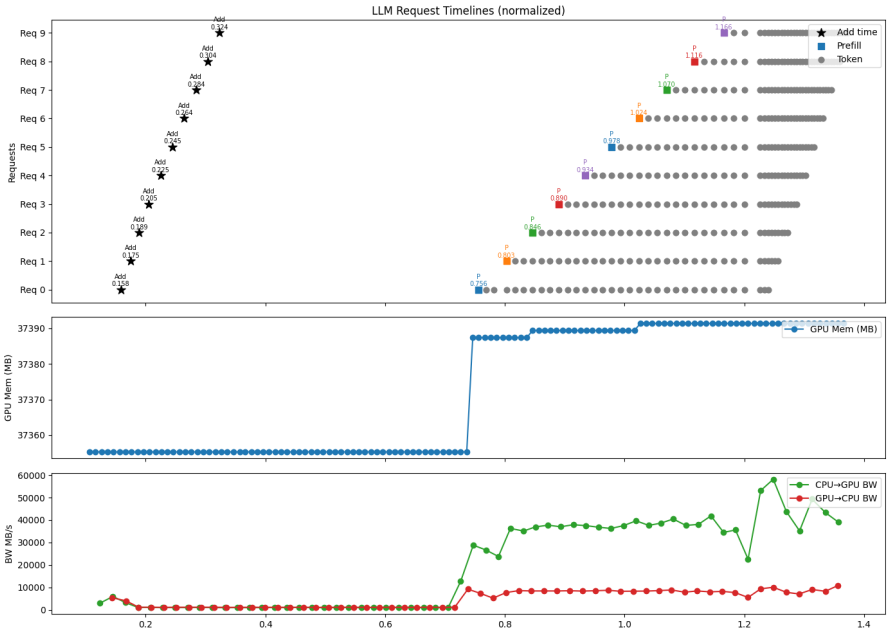


Figure 7.8: Chunked Prefill Enabled - 100 Requests per second

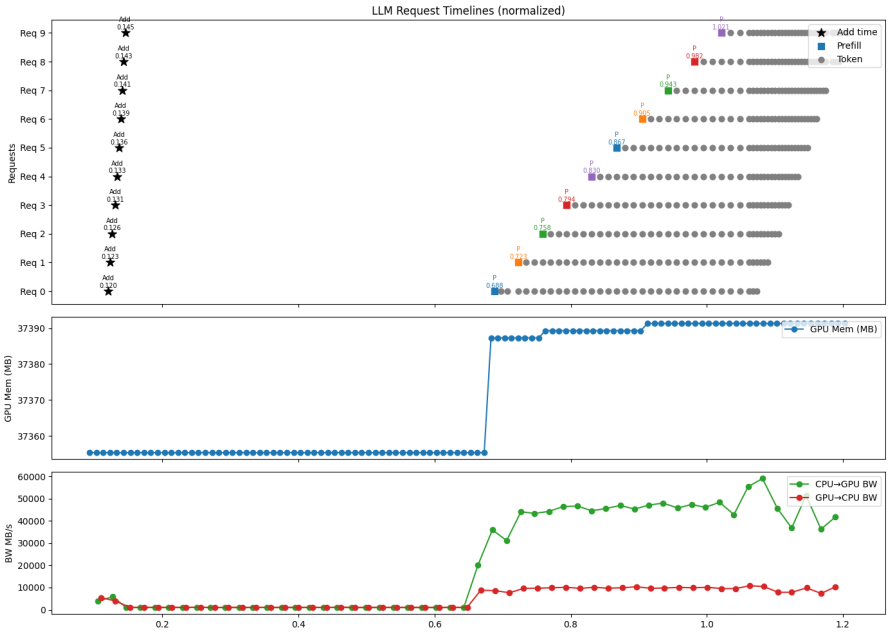


Figure 7.9: Chunked Prefill Enabled - All requests at once

7.5 Maximum Throughput with vLLM

All requests sent together

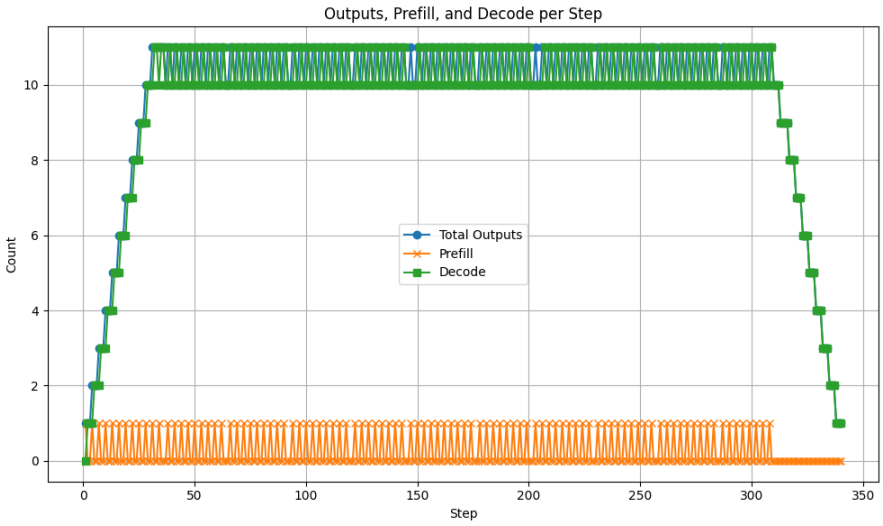


Figure 7.10: Number of batch outputs - chunked prefill enabled

Total time taken: 3.84s (100 requests)
Throughput Observed: 26.04 QPS

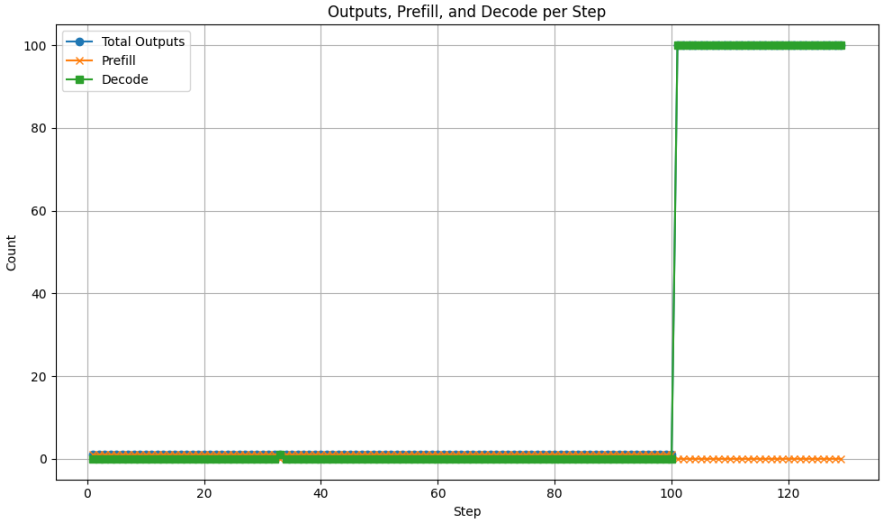


Figure 7.11: Number of batch outputs - chunked prefill disabled

Total time taken: 4.97s (100 requests)
Throughput Observed: 20.12 QPS

7.6 Nsight Systems GPU Metrics

Metric	Description
GPC Clock Frequency	GPU core clock (Graphics Processing Cluster). Determines SM execution speed.
SYS Clock Frequency	System clock domain of the GPU, affecting memory controllers, PCIe, and copy engines.
GR Active	Fraction of time the graphics engine is active (rendering or compute dispatch).
Sync Compute In Flight	Number of synchronous compute kernels currently executing (blocking).
Async Compute In Flight	Number of asynchronous compute kernels currently executing in parallel (non-blocking).
SM Active	Fraction of time at least one warp is active on any Streaming Multiprocessor (SM).
SM Issue Instructions	Fraction of cycles where SMs issued at least one instruction. Indicates pipeline utilization.
Tensor Active Instructions	Fraction of executed instructions that used Tensor Cores (e.g., matrix multiply).
SM Warp Occupancy	Ratio of active warps to the maximum possible per SM. Measures parallel utilization.
DRAM Bandwidth	Achieved global memory bandwidth (GB/s). Indicates memory throughput and potential bottlenecks.
PCIe RX/TX Throughput	Host-to-Device (RX) and Device-to-Host (TX) data transfer rates over PCIe.
PCIe Read/Write BAR1 Requests	CPU requests to GPU memory aperture (BAR1). Reads = CPU fetching from GPU, Writes = CPU writing to GPU.

Table 7.4: Nsight Systems GPU Metrics

7.7 Throughput vs Load Comparison For vLLM

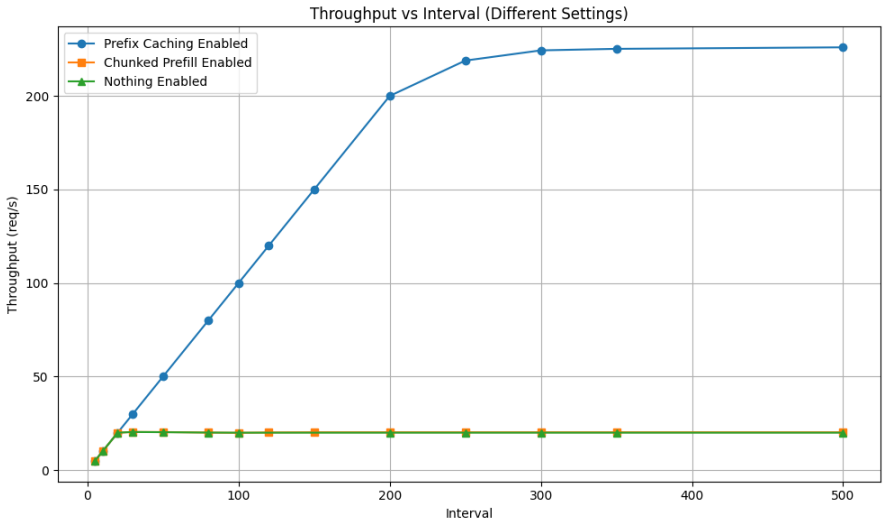


Figure 7.12: Throughput vs Load

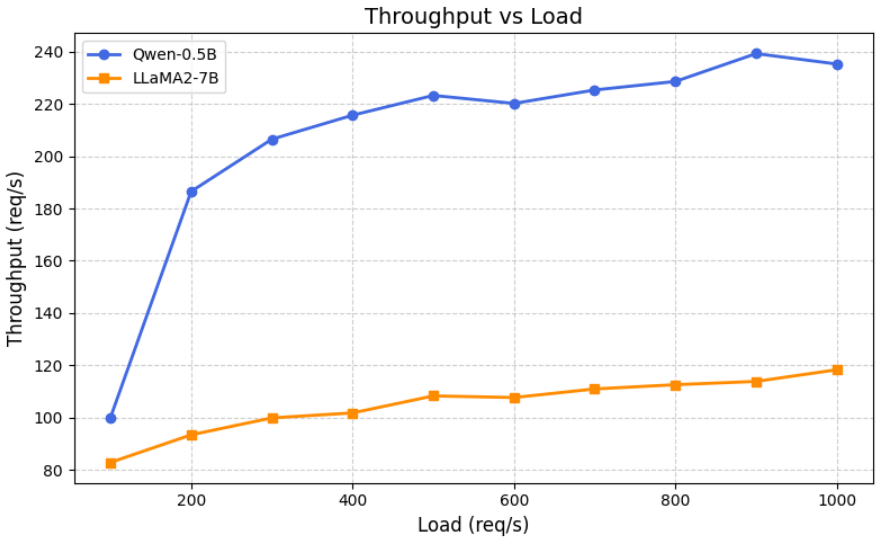


Figure 7.13: Throughput vs Load for different models for heavy workloads

7.8 Families of Requests

Model Chosen	10 families, each with 1000 queries	1 family with 10000 requests
Qwen 0.5B	294.228 req/s	308.750 req/s
Llama 7B	43.923 req/s	124.347 req/s
LongChat 13B	22.415 req/s	70.097 req/s

7.9 Compute Utilization across various max batch sizes

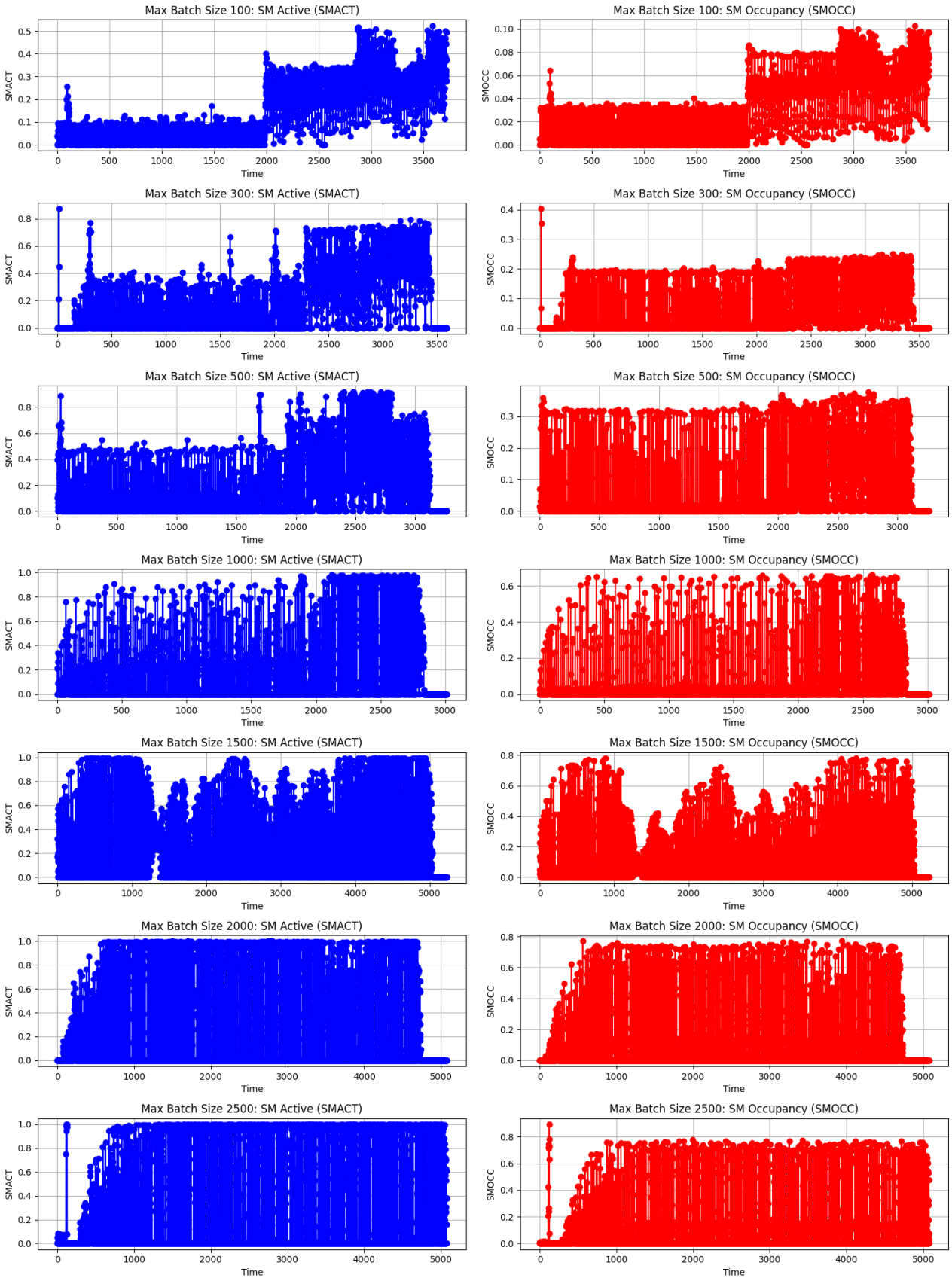


Figure 7.14: SM Active and SM Occupancy across different max batch sizes

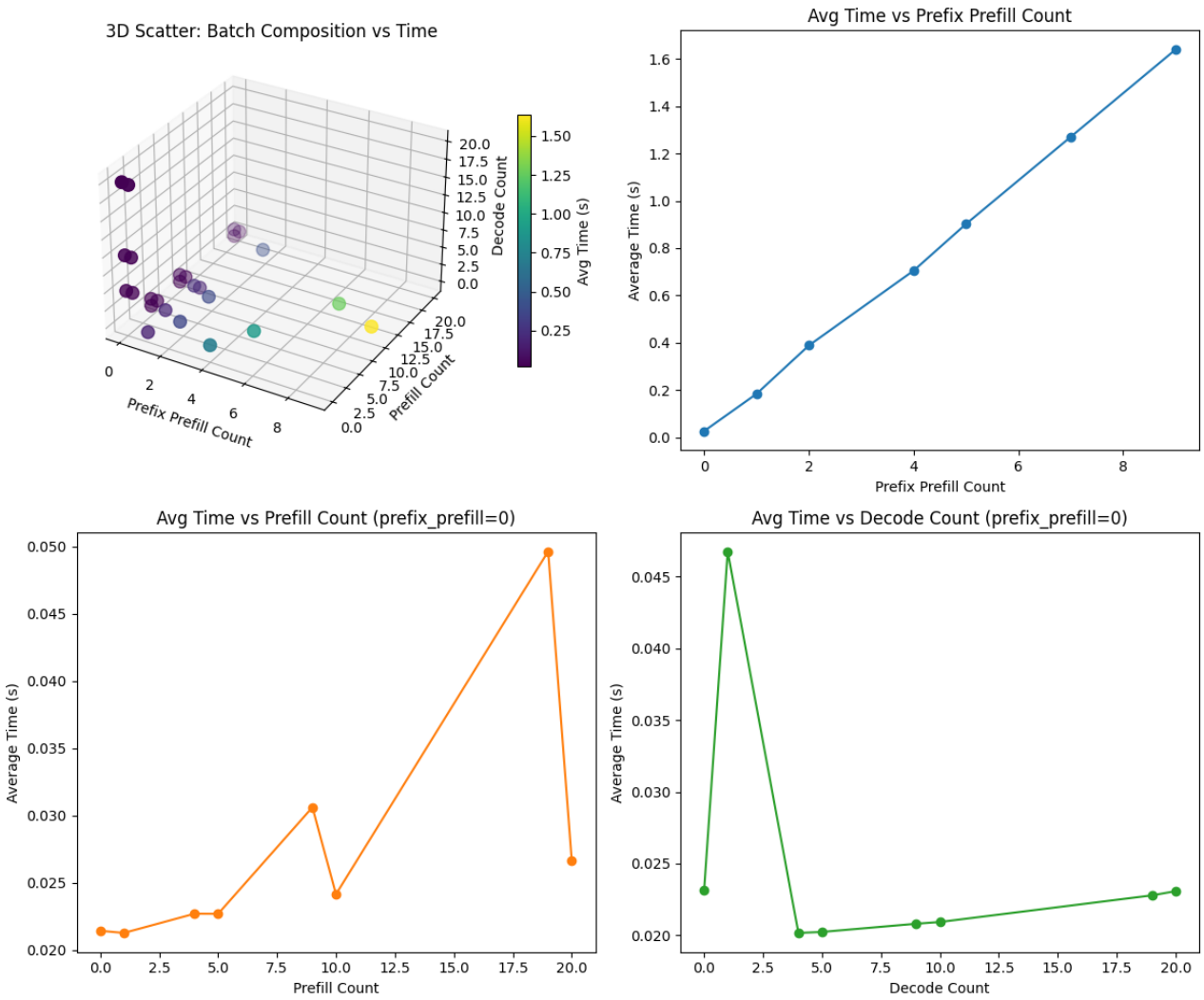


Figure 7.15: Times spent across different batch decompositions

7.10 GPU Memory Breakdown

Component	Meaning	Memory Occupied
Weights	Model parameters of the neural network. Static during inference.	0.93 GiB
Peak Activation	Temporary tensors generated during forward pass. Depends on batch size and sequence length.	4.07 GiB
Non-Torch	GPU memory used outside PyTorch tensors (framework/library overhead, buffers).	0.02 GiB
CUDA Graph	Memory reserved for CUDA Graphs to optimize kernel execution and reduce CPU-GPU overhead.	0.41 GiB
KV Cache: Intersection (Prefix)	Cached attention keys/values shared across requests (common prefix).	1,280 tokens / 0.0157 GiB
KV Cache: Extras for Requests	Cached attention keys/values unique for each request.	640 tokens (10 requests $\times$ 64 tokens) / 0.0078 GiB
KV Cache: Free	Remaining unused KV cache memory.	2,672,880 tokens / 32.8445 GiB

Table 7.5: GPU Memory Breakdown and KV Cache Usage

7.11 Merging queries belonging to the same family into a single batch

Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table 7.6: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

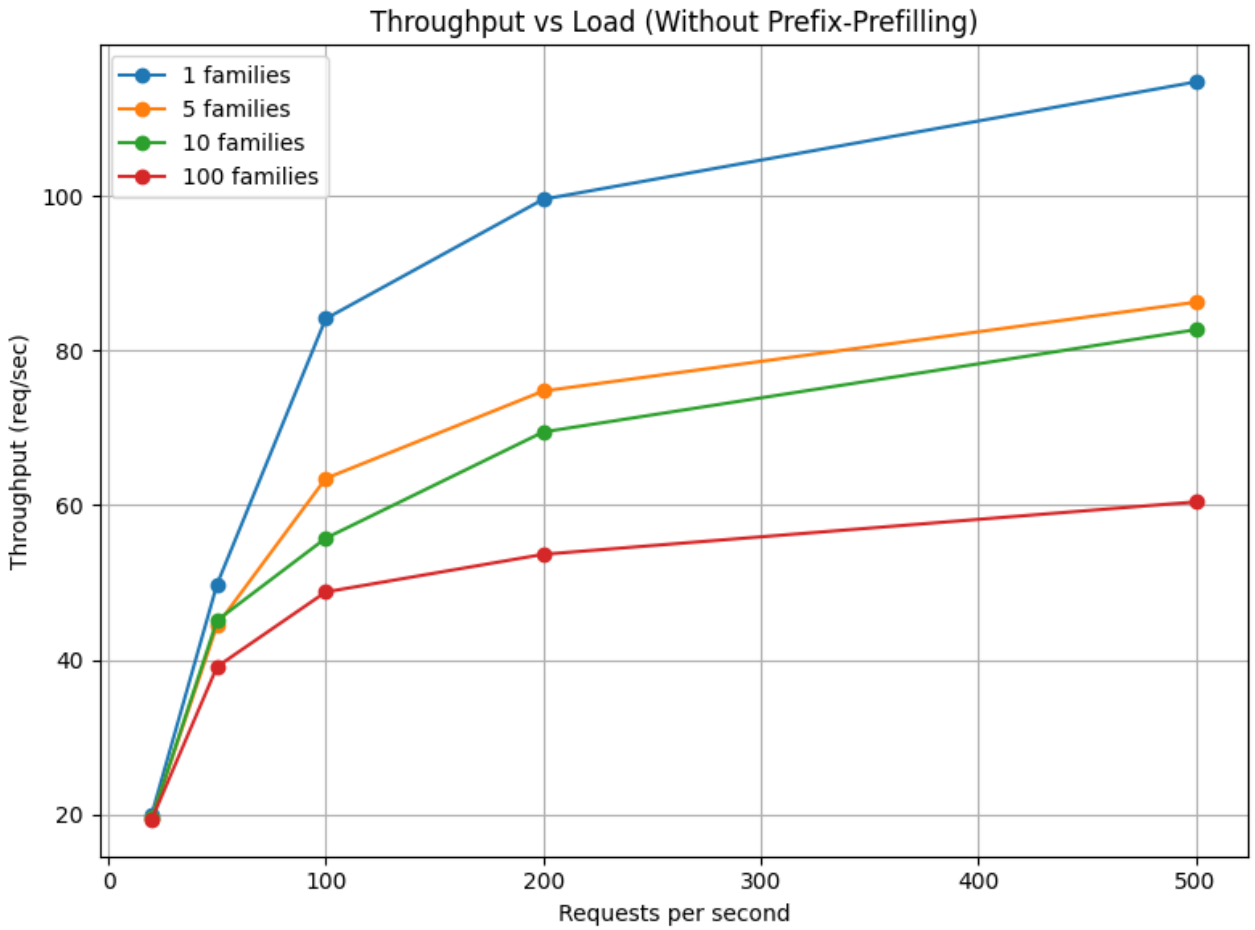


Figure 7.16: Throughput vs Load without prefix pre-filling

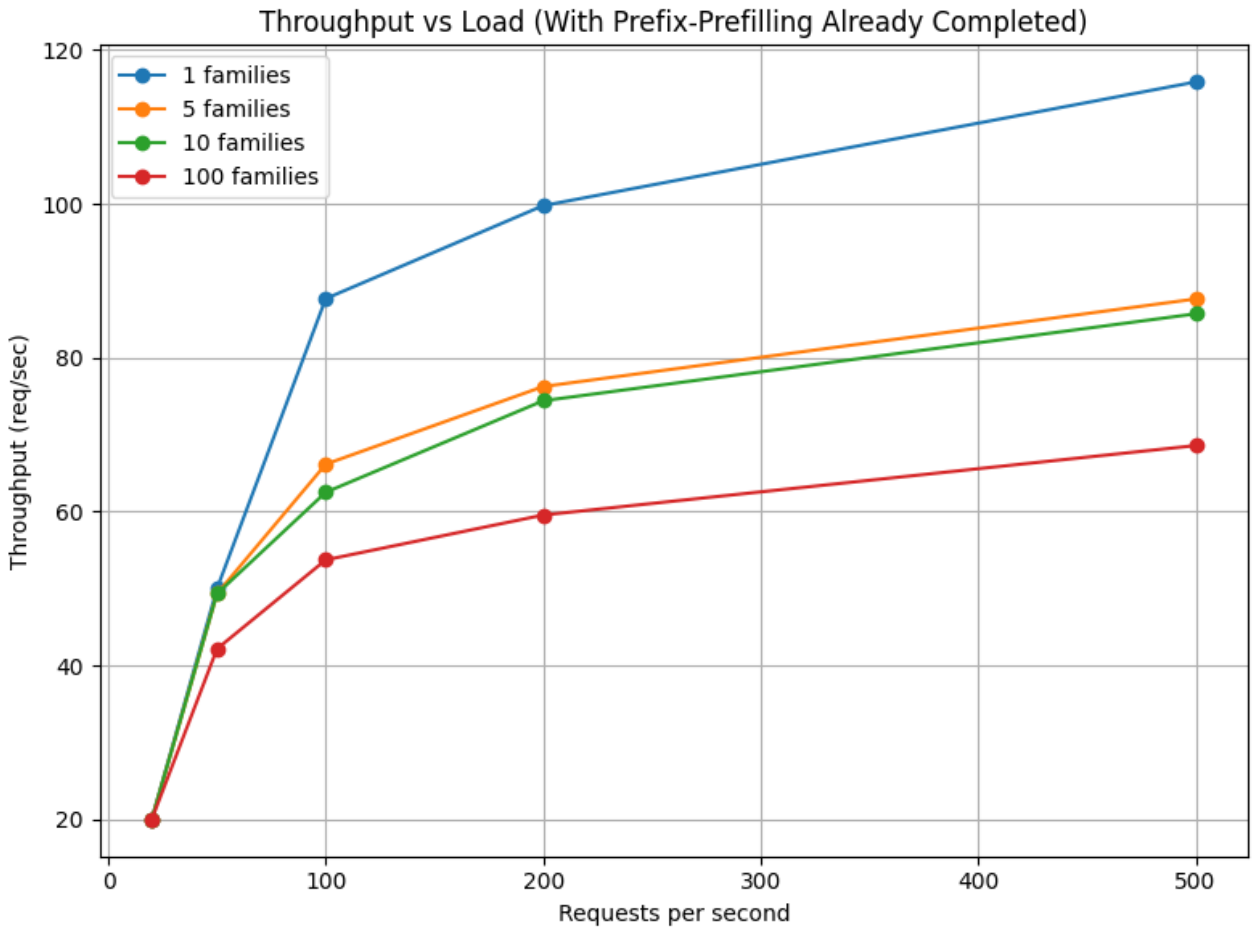


Figure 7.17: Throughput vs Load with prefix pre-filling

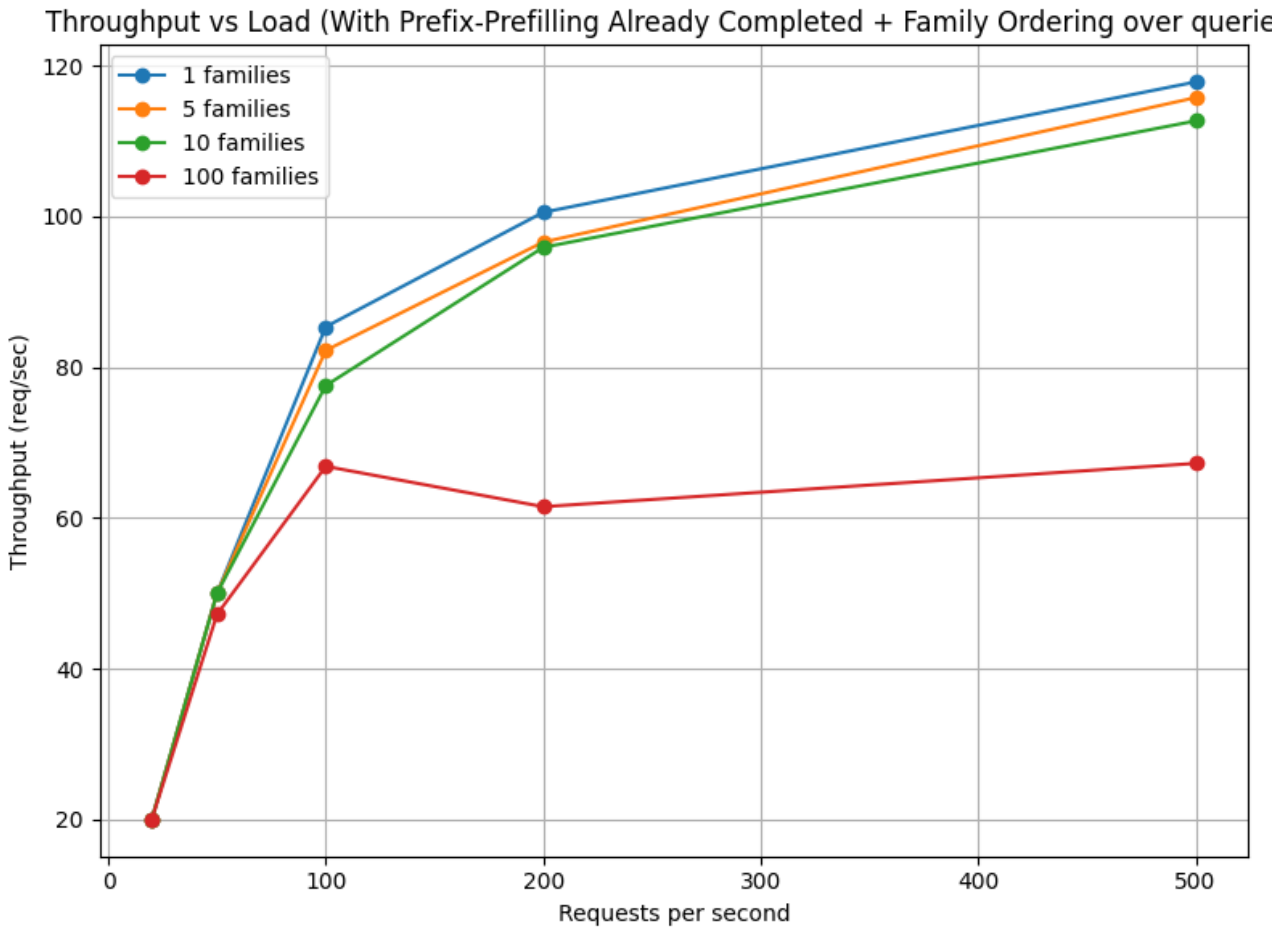


Figure 7.18: Throughput vs Load with prefix pre-filling with ordered family IDs

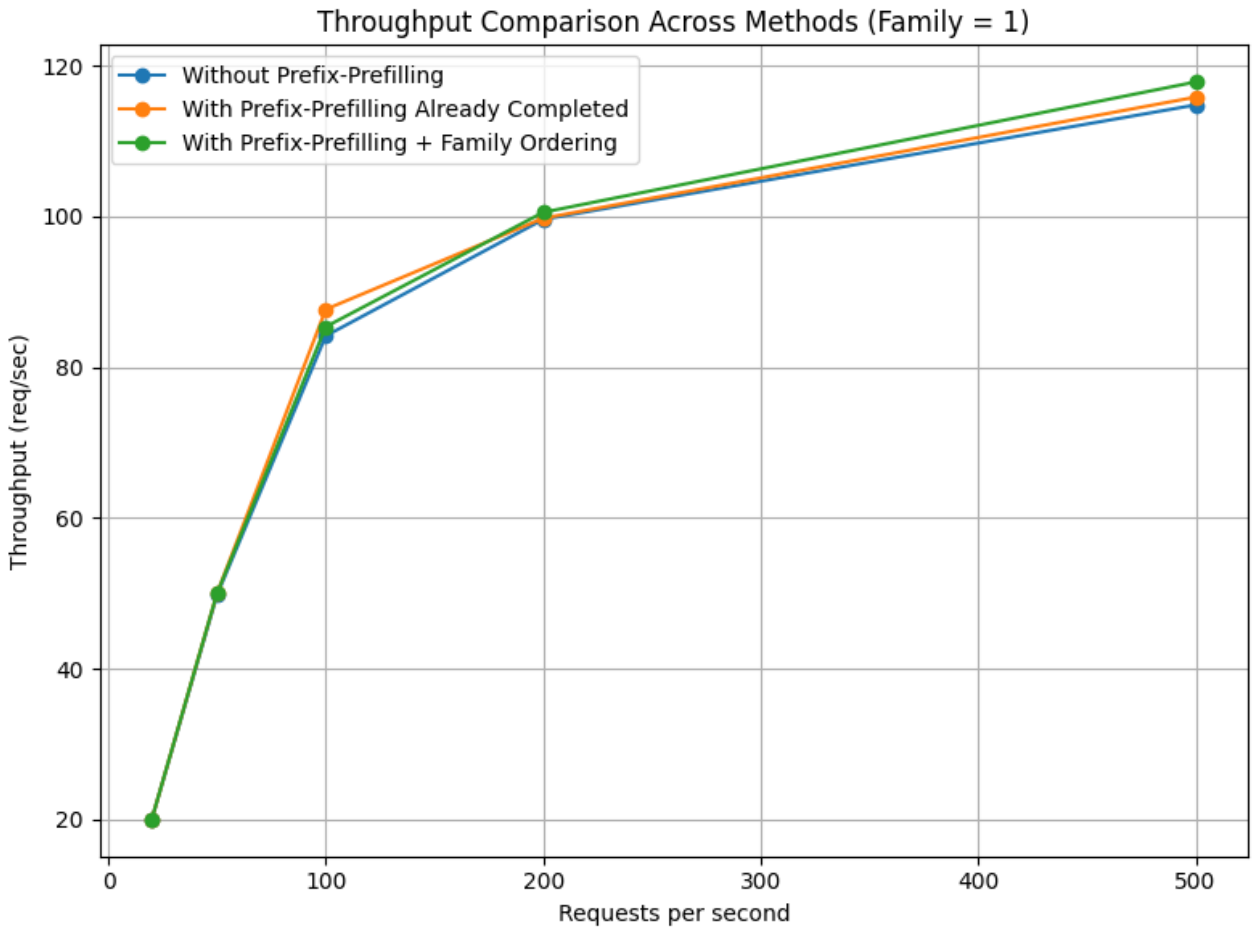


Figure 7.19: Throughput vs Load for 1 family

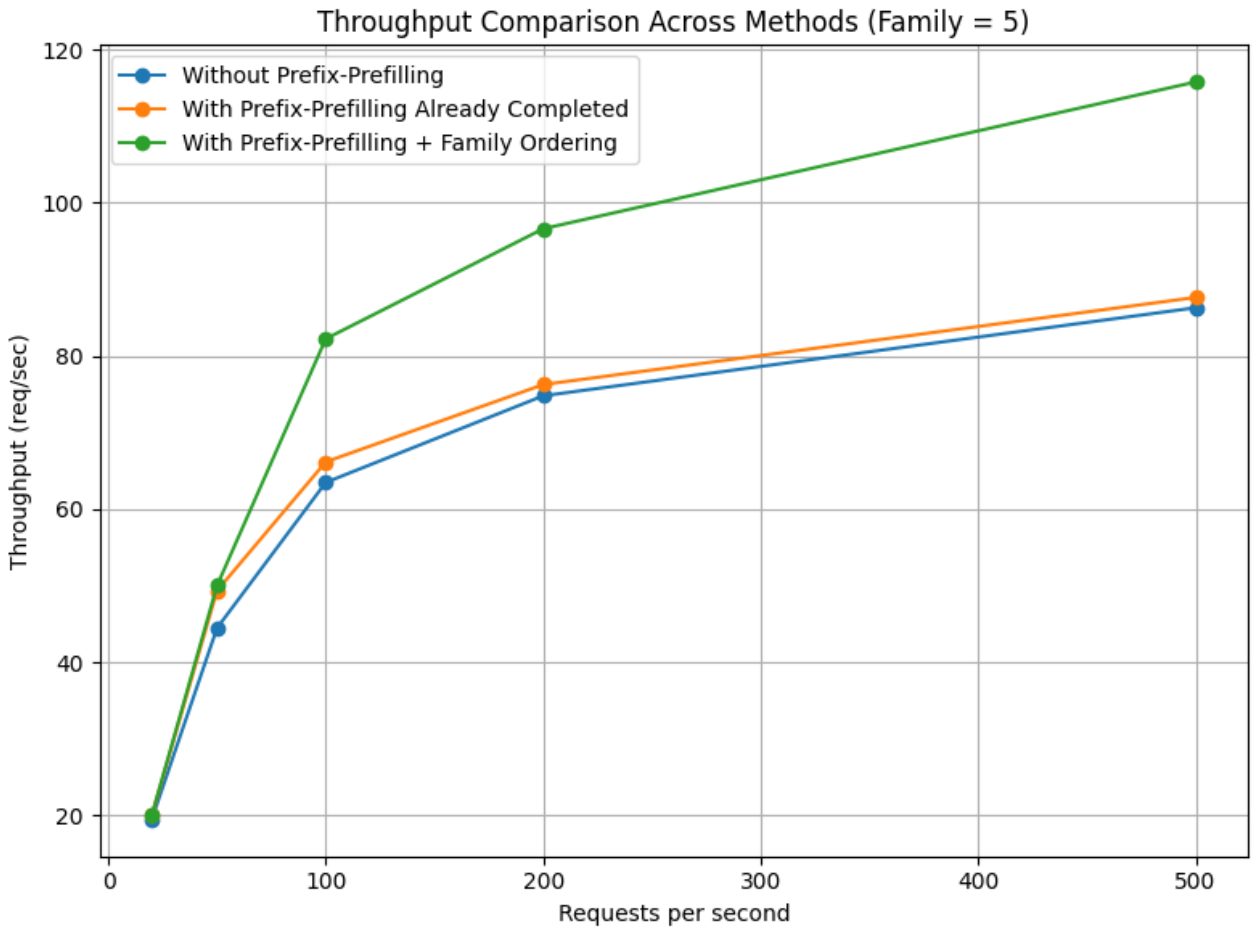


Figure 7.20: Throughput vs Load for 5 families

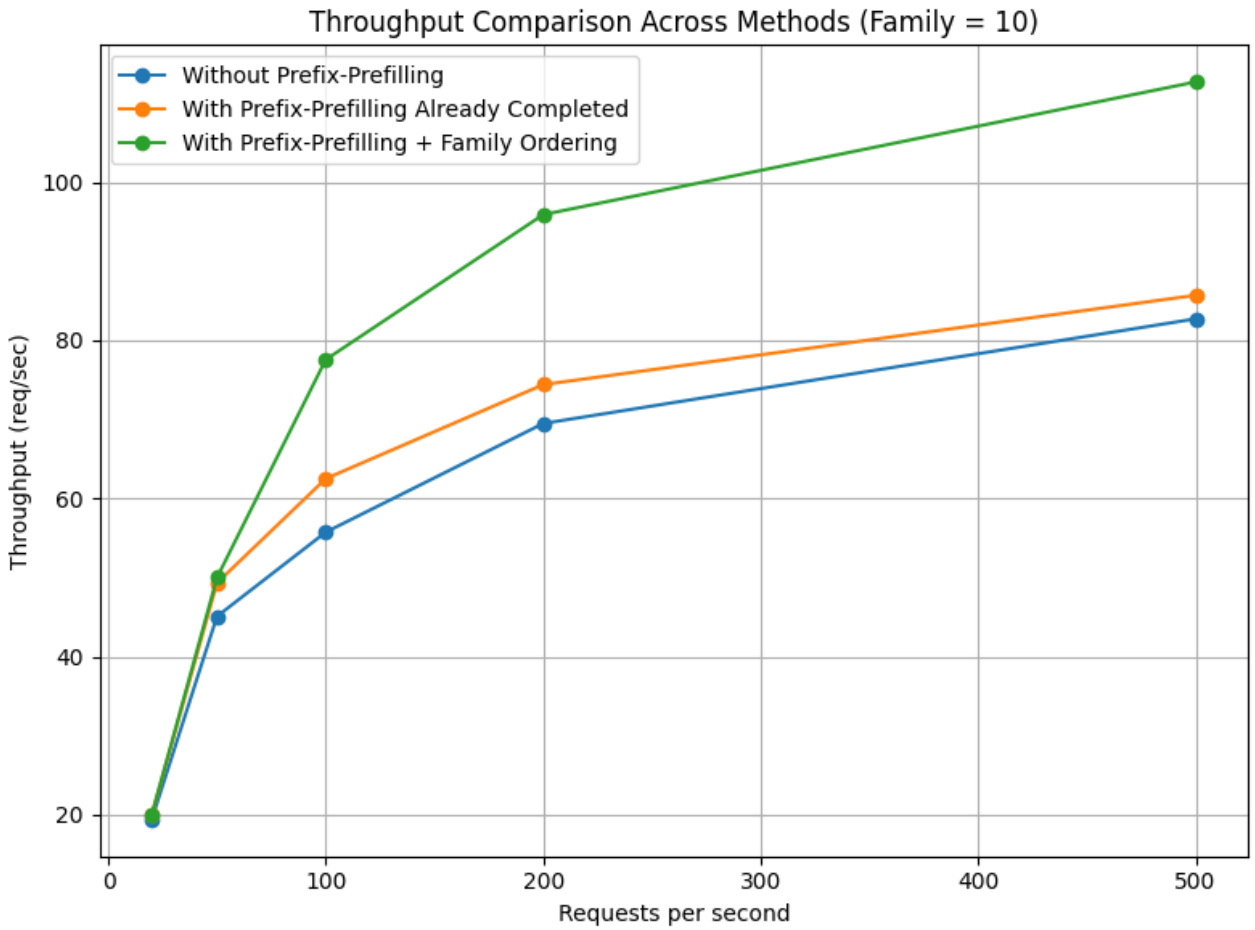


Figure 7.21: Throughput vs Load for 10 families

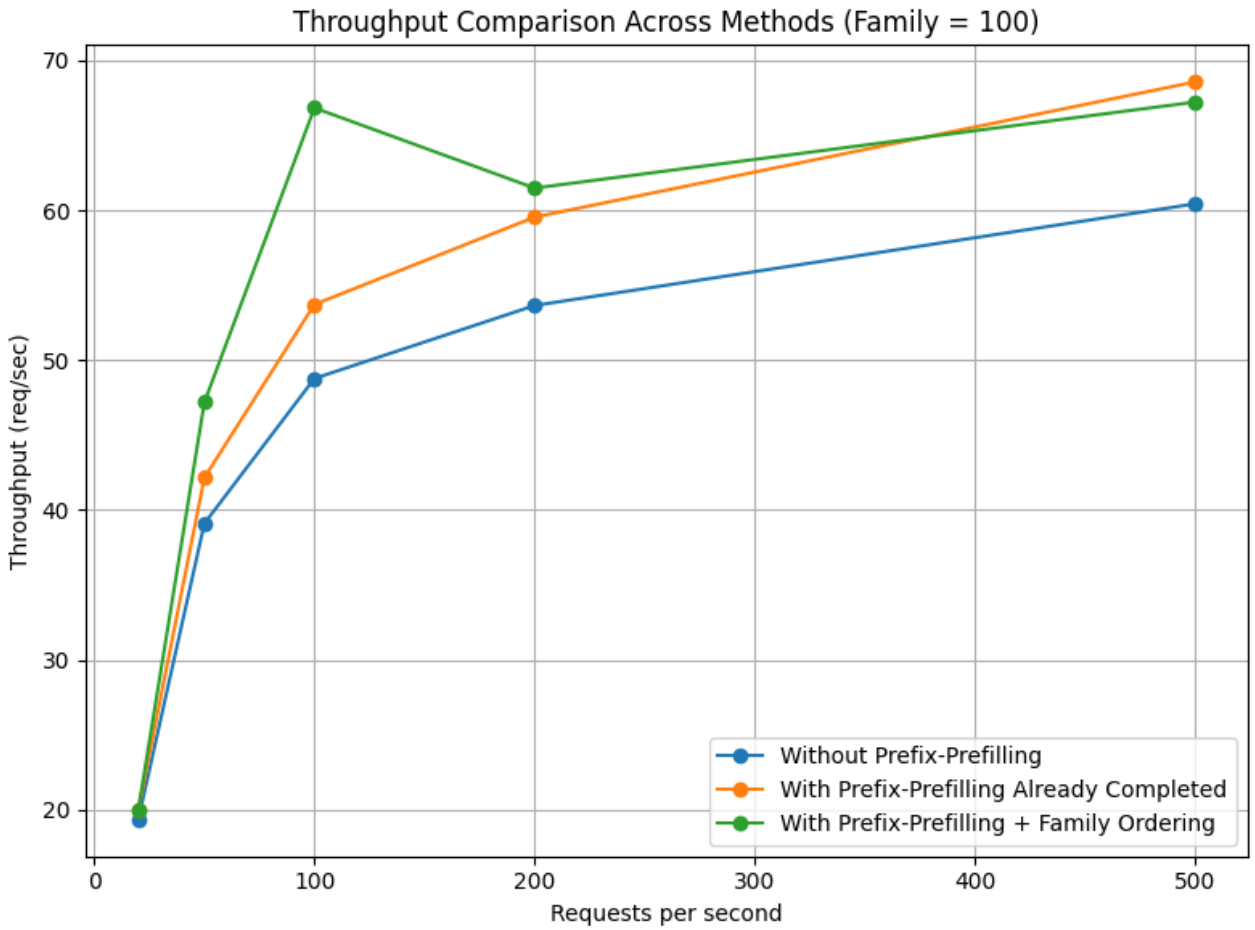


Figure 7.22: Throughput vs Load for 100 families

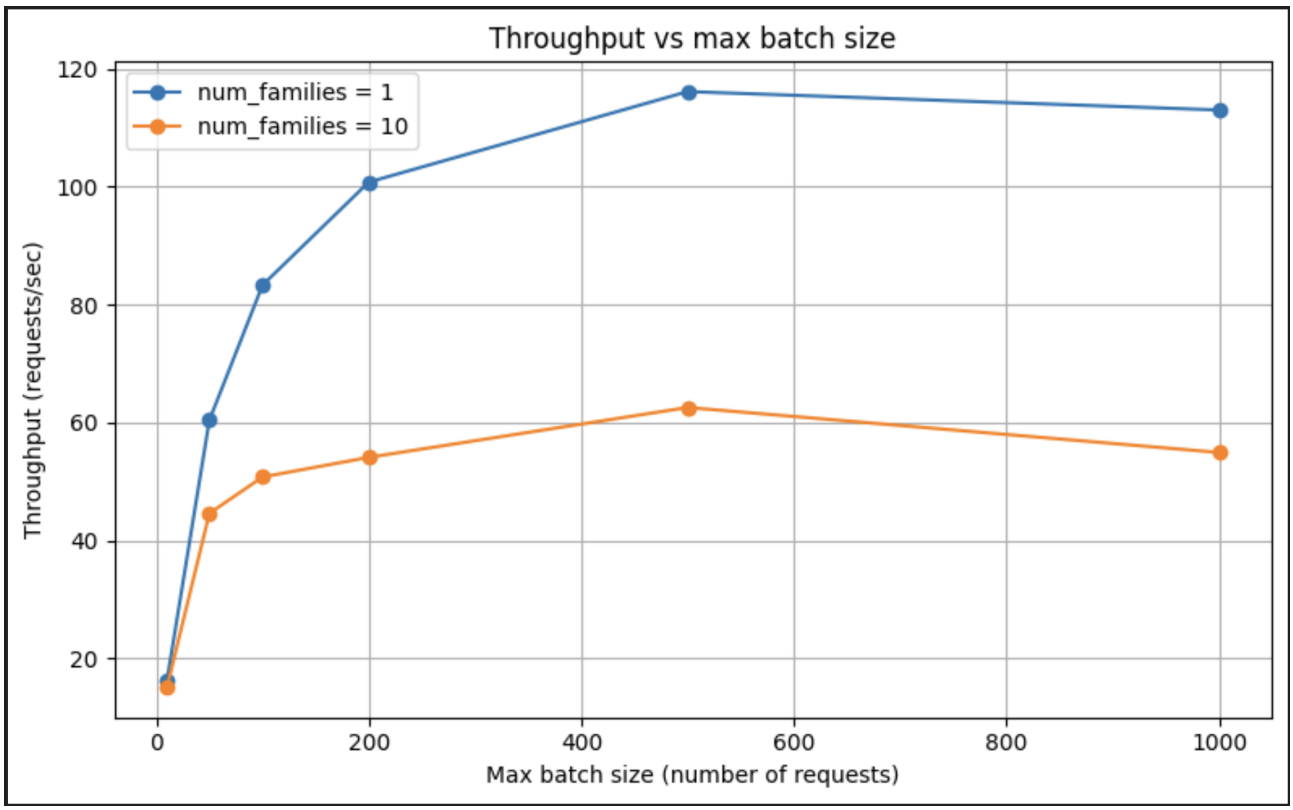


Figure 7.23: Throughput vs Max Batch Size for multiple families

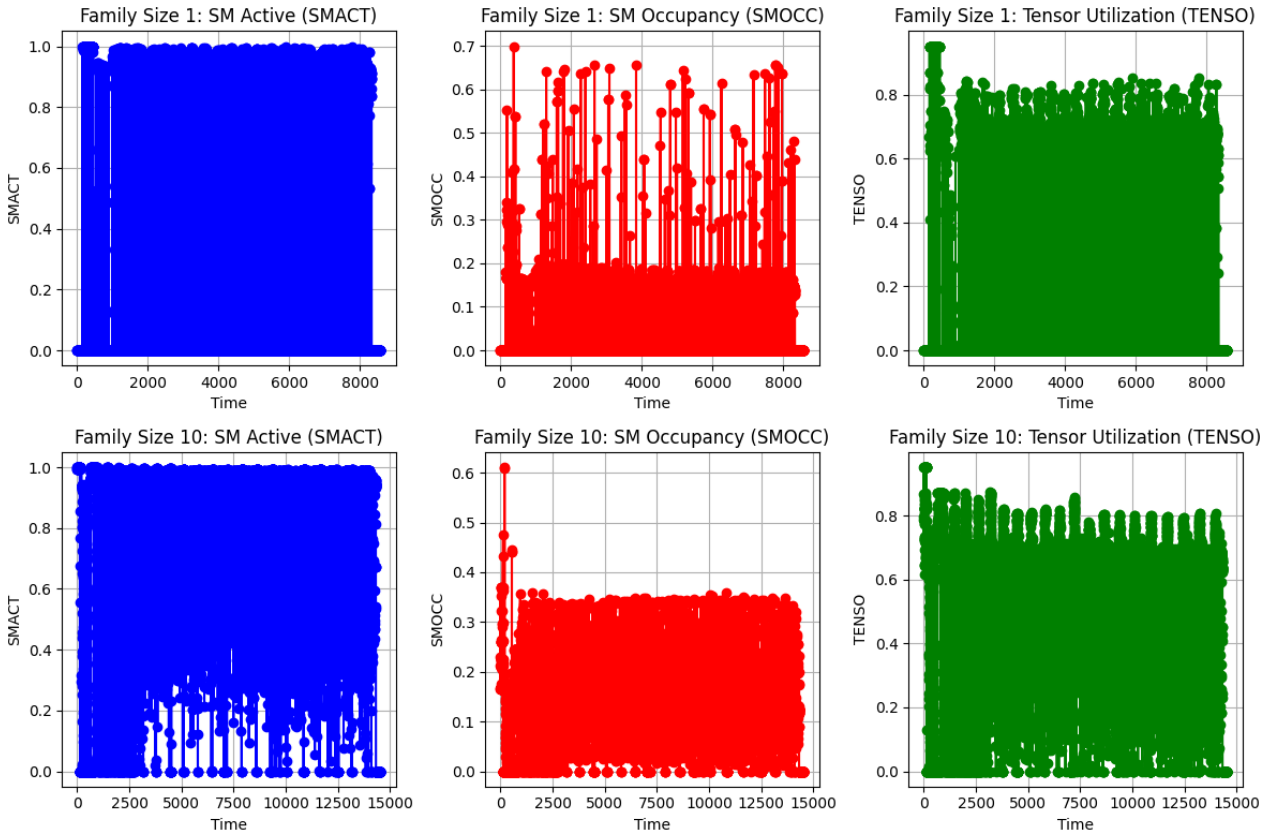


Figure 7.24: SM Activity, SM Occupancy and Tensor Core Utilization for different family sizes at max batch size 500

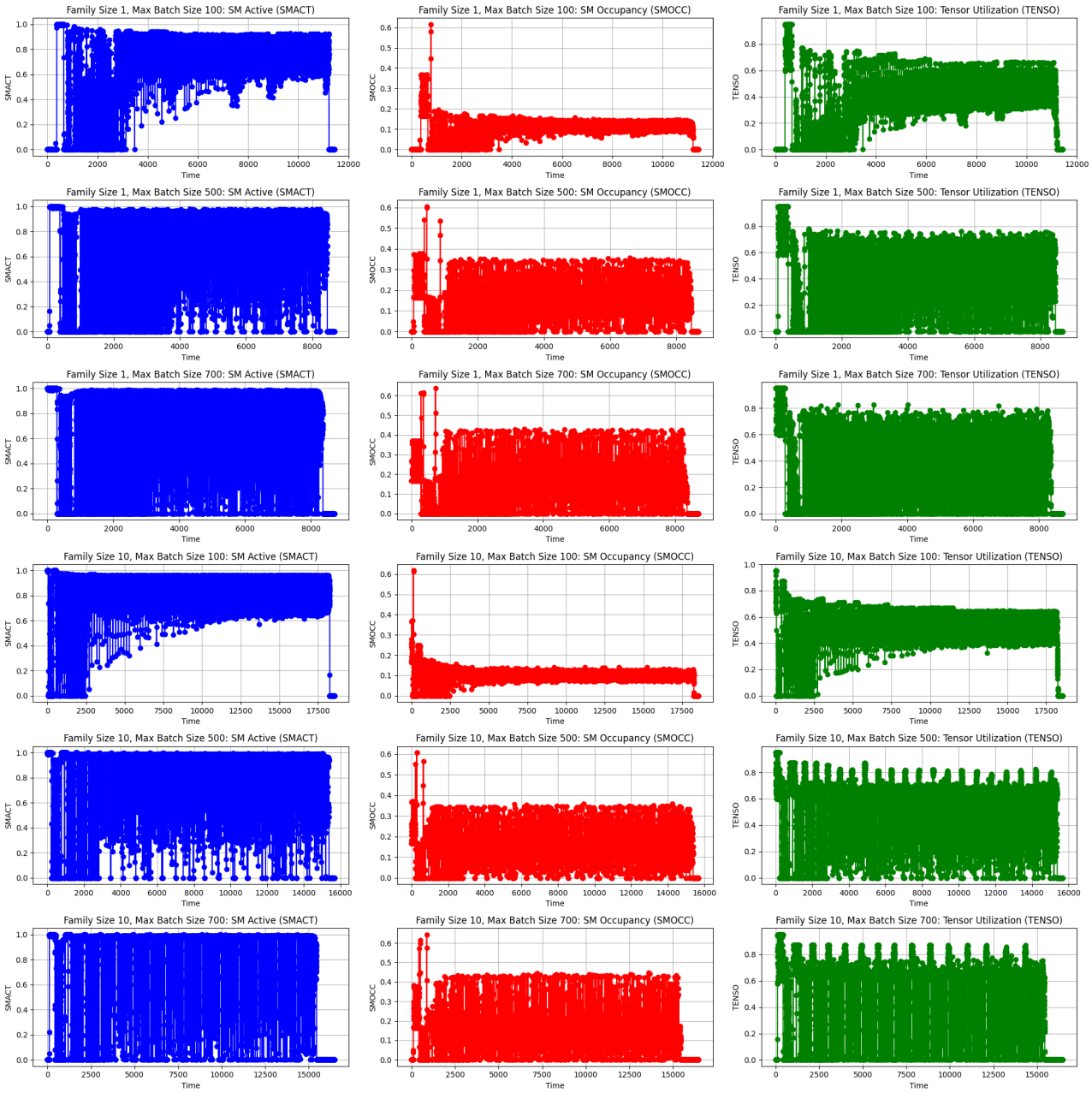


Figure 7.25: SM Activity, SM Occupancy and Tensor Core Utilization for different family sizes and max batch sizes

7.12 Comparison of vLLM with SGLANG

Number of Families	Batch Size	vLLM	vLLM (Family Ordered)	SGLANG	SGLANG (Family Ordered)
10	10	15.0942	15.5041	13.3730	13.8388
20	100	51.1423	54.7801	57.5661	62.3397
50	50	40.5825	44.5922	30.2049	37.0809

Table 7.7: Throughput comparison (in req/s).

Chapter 8

Maths behind LLMs

8.1 Transformer FLOPs

The motivation for this section comes from Casson (2023). Counting the number of floating-point operations (FLOPs) in Transformers is a useful way to estimate compute requirements and measure efficiency.

8.1.1 FLOPs Calculation

Let's explain each operation and variable here:

- Embed: learned token embeddings and learned positional embeddings. d_{model} is the dimensionality of the residual stream.
- Attention: QKV: linear layer in multi-head self-attention to project input into queries, keys, and values. n_{layers} is the number of layers. d_{attn} is the dimensionality of the output of multi-headed attention which is equal to $d_{key} \times n_{heads}$. d_{key} is the dimension of the key, query, and value projections. n_{heads} is the number of attention heads in a layer. In practice, Transformers are implemented such that $d_{attn} = d_{model}$.
- Attention: Mask: dot-product between query and keys. n_{ctx} is the context/sequence length.
- Attention: Project: linear layer to project concatenated attention heads output to d_{model}.
- Feedforward: two linear layers in the MLP block. d_{ff} is the size of the output dimensionality of the first linear layer. $d_{ff} = 4d_{model}$ is commonly used.
- De-embed: linear layer to obtain logits over vocabulary. n_{vocab} is the number of tokens in the vocabulary.

Combining all the non-embedding FLOPs terms gets us

$$C_{forward} = 2N + 2n_{layer}n_{ctx}d_{attn}$$

Operation	Parameters	FLOPs per Token
Embed	$(n_{\text{vocab}} + n_{\text{ctx}})d_{\text{model}}$	$4d_{\text{model}}$
Attention: QKV	$n_{\text{layer}}d_{\text{model}}3d_{\text{attn}}$	$2n_{\text{layer}}d_{\text{model}}3d_{\text{attn}}$
Attention: Mask	–	$2n_{\text{layer}}n_{\text{ctx}}d_{\text{attn}}$
Attention: Project	$n_{\text{layer}}d_{\text{attn}}d_{\text{model}}$	$2n_{\text{layer}}d_{\text{attn}}d_{\text{model}}$
Feedforward	$n_{\text{layer}}2d_{\text{model}}d_{\text{ff}}$	$2n_{\text{layer}}2d_{\text{model}}d_{\text{ff}}$
De-embed	–	$2d_{\text{model}}n_{\text{vocab}}$
Total (Non-Embedding)	$N = 2d_{\text{model}}n_{\text{layer}}(2d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{forward}} = 2N + 2n_{\text{layer}}n_{\text{ctx}}d_{\text{attn}}$

Figure 8.1: FLOPs Calculation Breakdown (from Casson (2023))

8.1.2 Model parameters (TinyLlama-1.1B-Chat)

$$
\begin{aligned}
L &= 22 \quad \text{(number of transformer layers)} \\
n_{\text{heads}} &= 32 \\
d_{\text{ff}} &= 5632 \quad \text{(intermediate / FFN size)} \\
V &= 32000 \quad \text{(vocabulary size)} \\
n_{\text{ctx}} &= 2048 \quad \text{(context / sequence length)}
\end{aligned}
$$

8.1.3 FLOPs Calculation for TinyLlama-1.1B-Chat

$$C_{forward} = 2N + 2n_{layer}n_{ctx}d_{attn} = 2.2 \times 10^9 + 1.84549376 \times 10^8 = 2.3845 \times 10^9 \text{ FLOPs.}$$

8.1.4 Efficiency Calculation

We ran inference on TinyLlama-1.1B-Chat using a single Nvidia RTX6000 Ada GPU. Based on the docs from Nvidia (ada), the theoretical peak performance for FP16 is 1457 TFLOPs.

8.1.5 Efficiency Calculation

We ran inference on TinyLlama-1.1B-Chat using a single NVIDIA RTX6000 Ada GPU. Based on the docs from NVIDIA (ada), the theoretical peak performance for FP16 is 1457 TFLOPs.

So, the time taken for a forward pass of $n_{ctx} = 2048$ tokens should be:

$$T = \frac{C_{forward} \times n_{ctx}}{\text{Peak TFLOPs}}$$

$$= \frac{(2.3845 \times 10^9) \times 2048}{1.457 \times 10^{15}}$$

$$= \frac{4.879 \times 10^{12}}{1.457 \times 10^{15}}$$

$$= 3.35 \times 10^{-3} \text{ seconds } (\approx 3.35 \text{ ms})$$

During our experiments, we observed that the actual time taken for a forward pass of 100 prompts, each with a context length of 2048 tokens, was approximately 4.4s. Therefore, the time taken for a single forward pass with a context length of 2048 tokens is approximately 0.044s (or 44ms).

Thus, the efficiency can be calculated as:

$$\text{Efficiency} = \frac{\text{Theoretical Time}}{\text{Actual Time}} = \frac{3.35 \text{ ms}}{44 \text{ ms}} \approx 7.6\%$$

8.1.6 Summary of the nomenclature used

Variable	Meaning
d_{model}	Dimensionality of the residual stream (hidden size of the Transformer).
L or n_{layers}	Number of Transformer layers (blocks).
n_{heads}	Number of attention heads in each multi-head attention layer.
d_{key}	Dimensionality of each head's key, query, and value vectors.
d_{attn}	Dimensionality of the concatenated multi-head attention output. Typically $d_{attn} = d_{key} \times n_{heads} = d_{model}$.
n_{ctx}	Context length or sequence length, i.e., number of tokens processed in one forward pass.
d_{ff}	Size of the hidden layer in the feedforward (MLP) block. Commonly $d_{ff} = 4d_{model}$.
n_{vocab} or V	Vocabulary size: number of tokens in the tokenizer vocabulary.
N	Total number of trainable parameters in the model.
$C_{forward}$	FLOPs required for a single forward pass per token.

Table 8.1: Variables used in FLOPs

Bibliography

Llmservingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale. URL https://arxiv.org/pdf/2408.05499.

Understanding the workload characteristics of large language model development. URL https://www.usenix.org/publications/loginonline/understanding-workload-characteristics-large-language-model-development.

Nvidia rtx 6000 ada generation. URL https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/rtx-6000/proviz-print-rtx6000-datasheet-web-2504660.pdf.

How llms generate text for the rest of us. URL https://pm.dartus.fr/posts/2025/how-llm-generate-text/.

Adam Casson. Transformer flops. 2023. URL https://adamcasson.com/posts/transformer-flops.